Gmail Uzair Afridi <uzairafrididev@gmail.com>

Inference from First Principles
1 message

Ayush Singh <ayushsingh718321@substack.com> 25 October 2026 at 12:28
Reply-To: Ayush Singh
<reply+3o5op5&3z4y8s&&91289f940f5128e3e82a7dcf121abba694fbbdc64f92366b90de580ff44380ca@mg1.substack.com>
To: uzairafrididev@gmail.com

Forwarded this email? Subscribe here for more

Inference from First Principles

AYUSH SINGH
OCT 25



READ IN APP

The AI SRE

You have a product and you want to develop an agent, and that agent will be some AI SRE. What happens anytime something breaks? Let's say that you're running some e-commerce company and e-commerce website. It's an open-source e-commerce website which you're going to use, and then we'll be deliberately making errors in that so that our AI can eventually get that alert and then eventually fix it. The idea is that anytime something goes wrong on our e-commerce website, it should be able to detect the incident.

And when I say detect the incident, it should be able to say what is wrong in this, and then it should be able to maybe gather the evidence of what eventually caused the incident. Then it should be able to detect possible causes. When I say that there can be multiple causes, so any breakage which has happened, it should be able to say, okay, who's responsible for it? Now, there can be multiple causes towards this, and then ultimately it should be able to propose a repair. I really mean it should be able to say that, okay, this is what I could eventually do. And then there can be some human

approval, and human approval is where a human sees that, okay, whatever the proposed repair, is it valid or not valid.

And then once the human says yes, it should be able to apply the targeted repair in a controlled environment so that it can also test whether whatever fixes it did really fixed the overall thing. Did it regress anything back? Or anything of that kind. And at last, it should be able to verify.

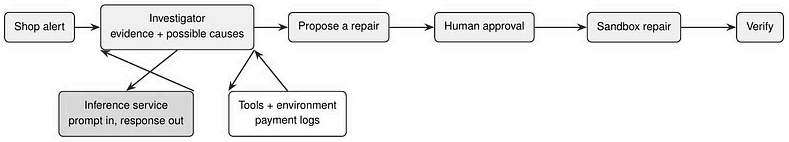
For example, let's say that there can be breakage of some user going, and then eventually they are not able to make the payments. Now there can be multiple things. There can be multiple users, tens of users who are not able to pay. There can be multiple alerting systems for it.

Whenever you design such alerting systems, you need to make sure what an alert is in this case. For example, you have latency increasing from maybe 20 milliseconds to 200 milliseconds. There can be multiple such instances, but anytime that threshold gets hit, after detection happens, there should be enough gathering of evidence. Once there's enough gathering of evidence, our AI SRE should be able to investigate what's wrong in it and then eventually go ahead and suggest some repair. And once it has suggested some repair, it should be able to say, okay, can I take a human approval? In this case, you can see over here that eventually it should take a required human approval.

And then once the human approves it, then eventually it goes to the sandbox repair, and then it should check whatever fix it has made, whether it has worked or not.

Whenever a checkout fails, there can be multiple possible explanations to it. Now, eventually you want your model to be able to use a proper investigator, and there may be a group of agents over there. Here, one of them proposes an explanation of what eventually went wrong. Then choose an observation, for example, whether the payment service is involved in the checkout failure, configuration is involved, or connection is involved. You want the right observations. And then another agent is making sure what is unknown, and there's another one who is saying what should be the next step. There can be multiple ways to eventually look at this.

The AI SRE



The model and the harness

My question is anytime an incident comes, what should my AI SRE be doing? Should it directly suggest the code or what should it do? First of all, analyze the incident. To analyze, it can also call some tools. Can you get me payment logs? If there is something wrong with the payment, then can it call some tool to say that, can you give me payment logs? Can it also call another tool to get some other logs which are related to it. It can also request information. Now request information how? By saying, hey, can you give me those tools? Can you call that particular tool? Can you call that particular access tool?

Once it has enough tools or understanding of whatever has happened, then it can propose a particular patch. And then after proposing a particular patch, it should be able to say, now here is the patch, call another tool that can apply this patch. There is a lot of involvement, not just one shot, the direct give me the call and then I'll give you the code that will fix it. It is not just one call. There are multiple calls involved to be able to finally reach the final output.

One is the model, which in this case is our LLM. Another one is my agent program, which is the orchestrator, in my opinion. Anytime the model is suggesting something, it is suggesting a set of text that, hey, can you call me the tool? Now the agent program's work is to make sure it is the engine, it is a harness. That's what we call as a harness, in my opinion. Over here we can call this a harness, or we can call this your orchestrator. Let's say the model said, can you give me the payment service logs. Here, the work of the agent is to call the required tool that is available to it.

Once the read tool returns some evidence, it should go back over here and the agent program should give it back to the model so that model can suggest the next step. It's a react loop in general. Over here, just to distinguish between what is your model, what is your agent program, and

what are the tools, environments, there are multiple things involved over here. There can be multiple environments, there can be tools, there can be policies.

There can be a set of policies which your agent has to adhere to. It can be an organization or organizational policy. It can be an environment up there. Then there are sets of tools which are available to it. These are the things which give access to your agent to the world. The agent program is the one which is eventually interacting with your model. Because do you really think your model can do some action? Can your LLM do some action? Just, so what it can do in general, it can either suggest that we can call this tool and it's the code which I want you to apply at this position. That's the maximum it can tell as text.

The agent program over here is the most complicated work we do, which is known as harnessing. Is how do you build such harness that interacts with the world at the same time with the model? To be able to do the work consistently.

There is a difference between what your LLM can do and what your agent program can do. Over here, there are 2 loops which eventually go in general. One is the model calls. Maybe it can request some tool. It is a more agentic stuff, which means your model will call something. Then your agent program has to give it access to the right set of tools so that it can get the right sort of evidence so that it can eventually read that evidence, give it back to the models, and then that harness is there. The harness which I discussed right now is it should be able to detect what's wrong over here.

It should be able to put out some, what are the causes related to it. It should propose some patch. Then after it proposes some patch, it should be able to say, okay, now let's take a human approval. If human approves that patch, then it should eventually apply that into sandbox environment. Then in the sandbox environment, it should test whether it is really regressing or not. That loop is what I call this application, which is a whole application over there.

It heavily requires product knowledge. Imagine you're developing some agent that can help you to get you onboarded. That is going to be on your website. And anytime a new customer comes in, hey, can I walk you through? Can I

make you go through the dashboard? Can I make you your own campaign, etc., etc.? You don't know how your visitor will interact with your agent, and because agent has to do the right sort of work and all. Now what happens because of that is you may define that, okay, your agent will be able to do XYZ capabilities, but suddenly anything which comes outside of it your agent is not able to do.

Not because if your LLM was not powerful, but because your harness was not generalizable to another thing. Because of that, what happens? Whole experience goes back to the agent. But the whole point of the agent was to make the experience good. If the user is repeatedly telling, hey, Can you do this? Can you do this? By the end of the day, user is going to go and then do it by himself or just schedule a human call. For example, when a human salesperson is onboarding a particular user, the human is understanding several nuances.

Engineering includes timeouts, retries, and distributed systems. Agentic engineering deals with issues created by your LLMs.

The inference service

Now you're giving it the input. You are an AI SRE. Here are all the tools which are available. Here is all the context which is available. Identify all possible causes. Your model's work is to generate the next token. Your model will say, can you give me the logs of payment service? Now, that is your model response. Your agent program has to understand that JSON file and then call. That is an application loop. But inside one model call, it's basically token-by-token generation.

You send that prompt to your LLM. Given whatever input prompts my investigator gets, it should be able to understand that whole context. It should be able to propose the next action, or it should be able to generate the next request to run something or the response.

Anytime the investigator, in our case is the AI SRE, which has access to your tools, which has access to multiple things, but it has access to one more service, which is an inference service. The work is to be able to give it, which is prompt in, and it should be able to also get the prompt out. This is the

inference service which we ultimately aim to design. But we'll zoom in exactly is what happens, which means what happens eventually whenever our model generates the next tokens. How can we make sure that we are doing it efficiently?

What we are focusing on is anytime your harness sends a request, model sends it back. You sent a prompt. In this case, it's a nice little alert. What has the evidence been so far? You send it to the model and model gives you something out of it. How can we develop that as a service? How can we make our model processing these alerts and evidence faster? And how do we make sure that a model outputting my result is also faster and efficient? From what angles are we looking? From the latency angle, we're looking from the cost angle, we are looking for the memory angle and quality angle. Whatever my model is receiving, how can I make sure that a model is able to process it nicely, and more importantly, from the several parameters.

Also, what is it outputting from taking in consideration of all these parameters? That is the service we ultimately want to develop, and that's, and that's what we want. That's what we call in general is the inference service.

vLLM: start the inference service

```
vllm serve Qwen/Qwen2.5-7B-Instruct --max-model-len 16384
```

Training and inference

Once you have a trained model, you need to be able to make it generate the next tokens. The ultimate work is once it decides, then it generates the several probabilities. Ultimately, what it does, given all the things, it tries to continue and complete your request. Anytime what we are doing is there are 2 differences over here. Training is what you have is eventually what is happening is your attention, your feedforward networks, and all of that is making sure that you have that right W's, the right weights up there. The inference is where you come to say, okay, now we have the right W's. Now let's use those recipes for which the model was trained upon.

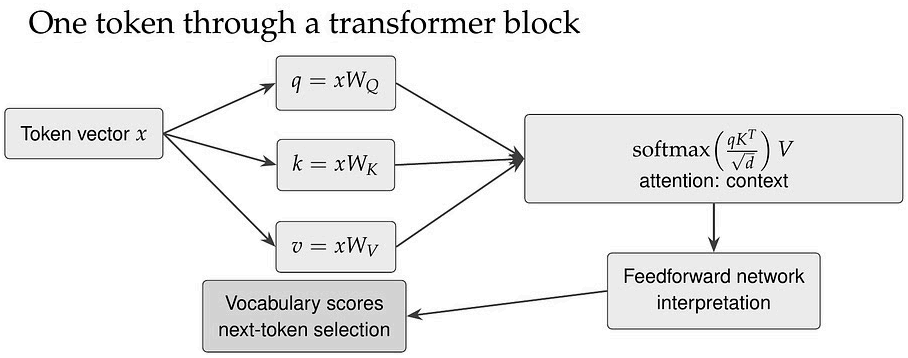
Now, over here, the one is the training and another one is the inferencing part. Now, inferencing part eventually means to say, now we have the learned weights. Can we go ahead and then now use the recipe in general? That's why the first question was there. Does inferencing change the weights or not? No, it does not change the weights. You already have the learned weights. Inference deals with predictions using whatever your learned weights are.

You have something like this, which is, now, the sky is dash. There can be multiple plausible completions toward them. There can be a particular next word, which is blue. Even in inferencing, your text gets split into multiple token size, which means using whatever tokenizer you want, you are going to convert that text into tokens. Every token is given an ID. What is this ID? And where does this come from? It's an address. Okay, this is an address from the lookup table. Whatever embedding table which you're using, it is going to be converting that. The 17 is, in this case, let's say your sky is converted from the embedding table, and then Just the example, it eventually has these nice little embeddings.

What is happening is once you have your token vector, first it goes through transformer blocks, and the transformer blocks have several abstractions. What are those abstractions? The first one is context. Context. Now we have the context too. Now we need to understand the context. That's what FFN means, the feedforward network. Interpretation. Right, we need to be interpreting what to do and then doing some actions. These were the 3 abstractions. If you go a little deep, we have attention mechanisms, then we have, the another one is the FFN. Yes, very good. And then finally, once you have the FFN, then we have the softmax to be able to give you the next token scores.

That is what we eventually do, even in the inference layer.

You want to predict the house price. You have already got good 2.5 multiply by some square feet You have got good weights. Anytime you want to make a prediction, you want to multiply with weights. Whatever the weights are.



One block shown. Each layer has its own learned weights.

Choosing the next token

You have 4 choices. Which one will you use? Which one will you choose? The sky is okay, blue. But must the model always choose blue? That is another question. If we eventually choose the token with the largest probability, it is eventually known as greedy sampling. Just greedy way of, whatever is you are looking at, which means you want to take the highest probability. What is temperature? What is temperature and how does temperature affect your output? What happens if temperature equals to 0? It will choose the highest probability answer. Whatever is the, high probability answer is. If your temperature is high, let's say I will say 0.8, then what happens? It becomes more creative.

It becomes more flexible. It may end up choosing sometimes other words, which is clear, dark, cloudy, whatever.

What is greedy? Greedy is always the maximum one. Let's say I say, check the payment logs. Greedy is always the maximum one. Top K is let's sample from the Top 3. Top K is one of the parameters. You say I always keep the best 3. You say top 0.9. What is top P over here? Top P says that anytime keep the best until they add up to 90%. Their cumulative probability is up to 90%. For example, this is going to be 50%, this is going to be 75%, then you add another one. This, till this time, the probability, the cumulative probability has already reached 90%. In top-k, you're only taking up to the top 3 words.

In top-p, keep the best. We'll leave the rarer words. We'll sample up to where the probabilities add up to 90%, because if you add another cumulative probability, these 2 as well, then it becomes 100%. You leave the tail of the

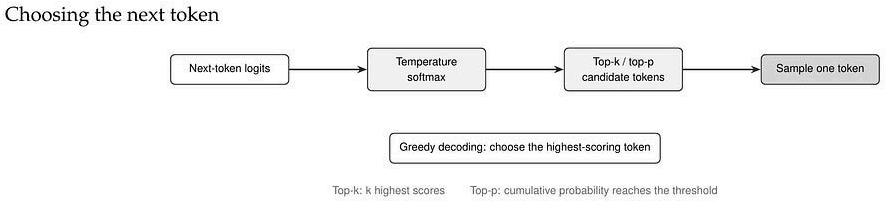
rarer words, and then you just choose from the, their cumulative becoming the 90%.

When you say top K, it will choose 50, but top P will only also do the selection from this 50 tokens. 50 will be chosen, but then top P will say let's just choose the cumulative one which adds. The selection choice will reduce to 20 tokens only in this point of time. Since the temperature is 0.7, What can happen is eventually the, there will be a good variety up there. You may choose all the 3 things. Yes, out of that only one token will be chosen. Maximum number of tokens. That's why you have max output tokens and all. Maximum number of tokens. And even the stopping criteria.

The best choice with the full setting temperature sometimes leads to a wrong answer. I'll tell you why. Why it leads to the wrong answer at times is if we are very strict with the model not being creative, We want it to be going always with the top answer. If that happens, then the model is also constrained towards what it otherwise believes. For example, let's understand this analogically also. The model said, check the payment logs over here. Payment, at least from the whatever it has learned, payment can be looked from there several other perspectives. Not just one perspective. Here in the context, if we choose the temperature to be strict, it will just choose the whatever highest probability it comes to.

But if you think about it, if you want some variety in this in terms of what words it should come next, especially in the creative field, for example, SBL, our temperature is set towards 0.7, 0.8, something like that. And but it tends to also hallucinate because of that. Right, but we want it to be creative when answering responses. We don't want to be strict, otherwise my response with SBL will give is very generic. But in the case of financial stuff or transactional stuff, you have to always be very strict and with additional guardrails. You can't be dependent upon it, but at least one way there is to set the temperature in a way so that you don't have to be creative enough up here.

That's why We have this temperature choosing. It's not a universal choice depending upon the application.



Sampler choices in the running example

Setting	Candidate rule	Example
Greedy	Highest score	temperature = 0
Top-k	k highest scores	top_k = 40
Top-p	Cumulative probability	top_p = 0.9
Repetition penalty	Adjust repeated-token scores	repetition_penalty = 1.1

Example settings are choices to evaluate, not universal defaults.

vLLM: greedy and sampled generation

```
from vllm import LLM, SamplingParams

MODEL = "Qwen/Qwen2.5-1.5B-Instruct"
llm = LLM(model=MODEL)
prompt = "The sky is"

greedy = SamplingParams(temperature=0, max_tokens=1)
sample = SamplingParams(
    temperature=0.8, top_p=0.9, top_k=40, max_tokens=20
)
for settings in [greedy, sample]:
    output = llm.generate(prompt, settings)
    print(output[0].outputs[0].text)
```

vLLM: sampler settings for different calls

```
from vllm import SamplingParams

tool_call = SamplingParams(temperature=0, max_tokens=200)
hypotheses = SamplingParams(temperature=0.7, top_k=40, top_p=0.9,
                            max_tokens=300)
```

```
report = SamplingParams(temperature=0, repetition_penalty=1.1,
                        max_tokens=800)
```

Autoregressive generation

In this case, we are saying, okay, the sky is blue. The sky is ultimately coming. We have produced one token. Now what happens? Now your next token will start to get predicted. Now again, the next token will start to get predicted, and so on. You're going to ask the same model to be able to do the next token prediction up here. Now the next token prediction can be the sky is blue. And then you want to ask the same model to be able to predict the next token. You want to get some word, then you want to have some things, and then you're going to use the same model to be able to predict the next word.

Absolutely not. You're not changing the weights. This is known as autoregressive generation also. If I'm correct, then this is known as autoregressive generation. Is eventually it is generating with the highest probability, but you can control the temperature. It depends upon you, Chetan, how do you want to decide. This is again the hyperparameter. Most of the times, if you want your models to be creative and using something out of the words, then you eventually want to give it more creativity. But if you just want a very strict And just choose the highest probability tokens. Eventually come and say that you control the bandwidth, Sudarshan, by the number of tokens it can generate.

Okay, so let me give the simplest Idea. It's basically saying probability of the next token given the current sequence. There is a fixed set of weights up in there. That is what we are ultimately talking about.

What is query, what is keys. The payment failed. Next, explain why it happened. The payment succeeded. Explain why it happened. We have both of the same words over here, but who is changing what? The context itself matters because when both of the things, the requests are same. Explain why it happened. Explain why it happened. But eventually, who's changing? Who is changing the whole direction? The payment failed and the payment succeeded. In this case, we may call this a context. And how do we get the context? We studied also that there are 3 pairs: query, keys, and values.

The sky is blue. If you are at is, then for is token, you are going ahead and saying, hey, what I'm looking for, 32 different ways. And then you multiply is token size with the query weights. With the 32 different recipes, you match against all the previous tokens. Then you combine all of that to get a final attention vector, the representation of your is token. Your u gets converted into a normalized vector. The normalized vector eventually gets converted into another h. This time h is filled with the context around it. H in this case is your final representation of the context.

Why does the first token take time?

You are an AI SRE. Your work is this: your model responded, call the payment two. Okay, call the payment two so that they can get the loss. It took three seconds to give you the first token call, and then it took matter of another one second to give you. All the messages. Why does that happen? Why did it really happen? You have noticed that initially, or whenever you talk with GPT, it takes time to give the first few tokens, but once it starts, it completes instantly. Why does that happen?

There are 2 ways either, and you decide it like that. Either you make the user wait, the design decision. I'll come to more of the KV caching and all, but more from the design perspective also. You make the user wait. What you do, you say that you first of all do all the answer and then you only show all the text. That time your user will say, okay, something is coming up. And there is stream generation, which means part by part you generate the first token, then second token, then third token, then fourth token, fifth, sixth, seventh. Before KV caching and all, it used to be like that. We used to be very slow in terms of what eventually is producing the right next response.

Similarly, this is a, purely a design decision where you're saying either you wait the entire time and then you just generate the final token, final all the text. But that really meant that why it is a problem, why buffered response at times is a problem. It is a good experience, but why it is a problem? We don't know. User will be in uncertainty if something is really happening or not. For example, if GPT is just too much spends time in the one phase, then does it really make sense? If you generate one by one, then eventually people have an expectation management also. It also depends upon the use case and interface.

For example, if you can have a better way to, manage that expectation, definitely. But that's a purely UI/UX design how we eventually differ that up.

But the bigger question is, we sent a request to the model, work happens, and when we receive our output, What was making us wait for 3 seconds to our first token and 1 second to complete the entire answer? The 3 questions for this section are what happens before we see the output, what keeps happening afterward, And what is fitting in the GPU memory? These 3 questions which you try to address. And I know a lot of you have already answered what happens before we see the output. And but I will ask questions for those people who have answered it. I'm very glad that you are able to get this right.

You send them a long brief. Your agent. What is that brief? Logs, metrics, configuration in a system prompt as a huge, context to your model. Like a prompt— we send a prompt to the models. In coding agents, sometimes I send a huge prompt. Similarly, let's say you send a huge prompt to the model. Model sends a short reply, check the payment setting. Do you really think that in this case model needs to do less work or more work to give me the final answer.

Prefill: processing the known prompt

I will distinguish transformer blocks and I will distinguish engineering separately. Two things are happening in the first place. First is the transformer block. It should go to every token, calculate its attention, calculate its FFN. Then eventually for this token attention and FFN for all the tokens will go in the batches. And then finally it will give the next tokens. That is what transformer blocks will take a lot of time. That's why first of all it has to understand the context. It has to calculate this Q, V, K, then it has to calculate your FFN, which is W up, W down, and then it has to do W combination. If it has multiple heads, then it also has to do W concat.

Multiple computations are happening and not for one token. It is happening at least, let's assume, 1024 tokens. Let's assume that it is around 1024 tokens up here in one single sequence. But here your transformer block has to do a lot of work.

Similarly, here also in transformer blocks, you have already got your weights trained. WK, WB, which is 32 recipes of asking questions, 32 recipes of representing answer, 32 recipes of combining information. Then you finally internal representation. 192 recipes of opening a particular token. Then there is a way to also come making it back to 32. Then finally giving the logic. All of this you have learned. Anytime there are around 1,024 tokens up here, let's assume that. All of that has to be multiplied with whatever we have trained. Then only it will make sense for us to understand the context. From that we'll understand the context. Lot of computations. That is why the AI alignment, AI interpretation, AI interpretability will come into the place.

Similarly, here lot of time eventually went into your long brief. When the next token arrives, check the payments. Then they did some mechanism. Something was KV caching. You assume right now is caching. Because you have already done a lot of computation, the later computation becomes much faster because you have already done the computation on your entire thing of on your logs, whatever your input from. It becomes much simpler to operate.

Can I say that reading 8 pages is already available to us so we can do it parallelly? Whatever, I can do everything parallelly there, but writing 8 pages I cannot do parallelly. I need to generate one by one because I don't have access to the future pages. There are ways to do it. I can speculate that I could write this and then I do something known as speculative decoding. But people may do or may not do that as a separate thing. But at least option number A is more established. Here I don't have to speculate. I can do it parallelly. I can read 8 pages parallelly and then I have to write only 1 sentence.

No, after that. You have to read 1 sentence, which is you will do quickly, but you cannot do write 8 pages parallelly. But you can read 8 pages parallelly if you have some superhuman natural ability, which computers actually have. That's what GPU enabled them. Okay, but anyways, no, A is not faster. We can make A faster. No, I'm talking about both are the issue. But which one has more control over? I can control A or B, but B I can also— industry has come up with saying that, for example, you're writing 8 pages, can you write it parallelly by speculating? Sure, I can speculate, but it's not so established. But definitely people are doing it, industry is following some of their standards to make even generation faster.

But generation is also usually faster if you do the work on it. Yeah, so when I say control, I really mean to say the work to do so that I can speed up my time. It's specifically the time.

You can call this is processing the known prompt. We call that phase as a pre-filling, which means to process the available tokens, the prompt tokens, the context, what is already available to us. It's what is what we call the pre-fill phase. To generate the new tokens. We have things like decoding. Sorry, the later steps are decode. Token 1, token 2, token 3, token 4, token 5, all the way around, whatever is the max tokens which is allowed. Usually your vLLM kind of people handle this prefill and decoding step altogether. I'll just explain during the prefill phase so that it makes sense. Let's say that you have a token such as the cat is sitting.

Okay, now over here, your sequence. This is one sequence, that is eventually converted into x1, x2, x3, x4, x5, x6. 1, 2, 3, 1, 2, 3, 4, 5, 6. Yeah, 6 tokens. As of now, I have just made my own token in my mind. This is what the sequences are. Everything gets converted into an embedding, and eventually it eventually comes in. I have My token representation. Finally, my this filter goes through your attention. I call this context. It goes through FFN. I call this an interpretation. And finally. You have H, which is your representation, and then finally you have H multiplied by W vocab to get your set of scores.

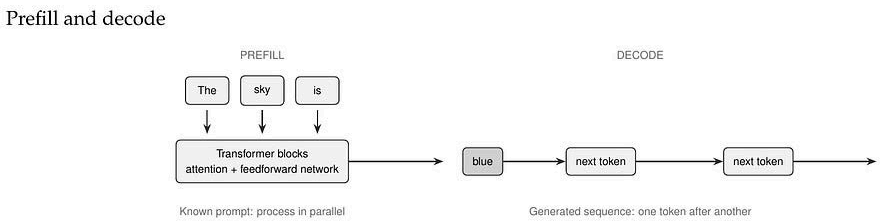
This is what is happening during prefill. You already have your weights ready. You already have your FFN ready, FFN weights. The W vocab is also available. What you're doing is finally getting the scores up here. That is how it is able to understand your entire context for all the tokens up here.

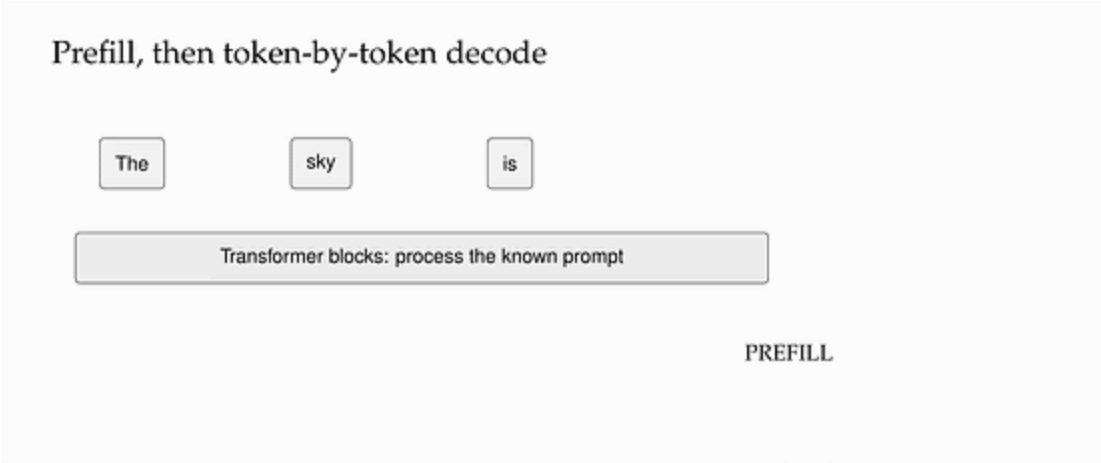
You will understand what I'm trying to say. Is basically I'm saying that prefill phases Going through the entire transformer block. I am not including engineering stuff over here right now, such as timeouts, retries, connections, setting up workers, queuing, GPU management. I'm not putting that thing in the account right now. Otherwise it will not make any sense what I'm talking about. Otherwise this attention block, when I'm saying attention block, what exactly is happening? It won't make any sense. What is happening is every time your X in this case is then converted into 6 tokens, prefill space is first of all is understanding the context. Then it is doing an FFN work where every token's representation gets converted into, in this case, let's say X to U.

U over here is having the representation of the context, what they mean to each other. Then you have H. In this case, H is interpretive stuff. Then H gets multiplied with the W vocab of the whatever the vocabulary that will come out for the rest of sports. Then decode says, let's pick up the first topic. What is the highest probability? Greedy sampling, or maybe, your autoregressive generation. Get the next token.

It is happening parallelly. What is happening here? You have x1, it cannot look at other tokens. Attention is happening over there, but at the same time, parallelly, this token is being processed, this token is being processed. It is looking at the previous one, but it's not looking at the Prior one. Attention, feedforward networks, everything, they all are happening parallelly itself. They are not waiting for one token to understand the context and going to the next token, then second token, then third token, then third token to fourth token. First of all, let's calculate attention here, then calculate. No, you see that transformer visualization, it will give you more clarity. That talk is happening.

Not sequentially. Otherwise, what is our use? I'll go back to my LSTMs.





Decode: generating one by one

If you have long input. In this case, which phase will have higher time throughput? Prefill phase will have more time, or will take more time, or decoding phase will take more time? Long input, long input tokens into prefill. Why does prefill take a long time? All the tokens, all attention, multiplication of the weights, FFN, whatever weights were there. Then you have final W vocab. Too many calculations. Complete transformer block, correct. By the way, this is one block. It has another 36 blocks in when. Transfer. This is just one block. You have another 36 blocks depending upon your transformer architecture. Every block should do its own calculation, then only something comes up.

That's why prefill will take a lot of time. And because attention made it very powerful in terms of, parallelization, long input eventually here, the prefill phase is going to be high. Prefill phase will take a lot of time. That's why over here, if you have a longer input, prefill takes much more time. And if a shorter output, decoding will be much faster.

Okay, what if you have a short input and long output? Decoding. Okay, why decoding will take a lot of time? It has to write 8 pages. It has to write something. It cannot do it parallelly. Speculative decoding? Let's not go there. But here, if you go with the formal decoding, yes, it will take time. It will need to write.

Let's say that you're building this AI SRE. Why prefill and decode will matter every single time? At every single LLM API. Think about it. It's very fun. Let's say that your agent has to look out an entire context. Then you will be able to classify a child. It takes this much amount of time to do the prefill phase. Can I do instead one thing? Instead of giving the entire logs, can I summarize using smaller models and then give them a shorter output, shorter inputs so that I can improve my latency, improve my prefill phase? That is a strategy. That is a pure strategy. You can choose to summarize before giving it to the model.

You can choose to do a lot of stuff there. You can choose to improve your RAG-based systems. You can choose to do a lot of the work if your goal is to improve the prefill phase. Now let's say that if your goal is to improve the

decoding phase, it's much harder because you have to completely speculate something known as speculative decoding. However, there are approaches to it. Similarly, I want you to understand the importance of it. What is the importance? Is not understanding what is prefill and what is decode. Is to understand, if I understand if prefill is the problem, for me, much easier job is when you work with Claude Code or Codex, you should be clearly distinguishing that prefill takes this much amount of time.

Your queue should be instructing Claude code for every LLM API call. Can you create a separate dashboard for LLM API call? It is telling you what is prefill, what is decode timing. Then you can go and say, okay, now that's where you will. Otherwise, if you have Claude is there, Claude will not do anything about giving you the cost matrix or latency matrix. It might do the tremendous work, but still you won't get to know that why I'm using that strategy altogether. Why I'm using prefill strategies such as prefix caching, KV caching, multiple strategies are there, right, to fasten up the prefill phase.

Prefill and decode: three workloads

Input tokens	Output tokens	What changes?
1,024	32	Baseline prompt and short answer
8,192	32	More prompt-processing work
8,192	128	More prompt work and more sequential output

Workload comparison; no timings are claimed.

Time to first token includes infrastructure

Prompt tokens can be pre-filled, can be done together, but future tokens are not decided. That's why decoding is up there. But only does the transformer blocks are involved in time to first token. Is only that thing involved, or do we have another thing? I just stop at another, you expand it. But tokenizer, let's put it in a transformer block. Infrastructure. For example, say for example, your engineering stuff in this case, you already have thousands of users. Queuing mechanism based on your infrastructure of how your stuff is. And for example, you must have your GPU compute. Basically GPU can only take a certain request at some point of time. In that case, You'll have to make sure that the viewing mechanism is great.

For example, you also have to do some prompt work. I told you. Instead of sending a huge prompt, you summarize the prompt first. You summarize the specific context. All of that will come in the infrastructural cost. Infrastructural time. Your time to first token is not just about your memory layer, sorry, transformer blocks. It also has to deal with your infrastructure layer.

There are a lot of infrastructural layer which is involved. For example, we'll end up using some managed RAC sources. Assume it like this. We will use some managed RAC service. In that we have to pull up the top case from our code base. That we have to retrieve the top K and then put it stack in the entire prompt. And then we ultimately have to make sure that things are proper. If our RAG service provider is down, then that pieces will be not there. That will impact our entire infrastructure. Those are engineering decisions. When we'll design the agentic patterns, we'll come there. As of now, I want you to realize that these are, these are the pieces that cannot be ignored.

I'm just talking about one inference layer. But there is another layer also, which is the infrastructural layer or other components. For example, let's say in this case. For example, let's say that you need to send them the logs. It needs to get retrieved from somewhere. A tool is not working. A tool is some parameter has been done wrong. There can be multiple instances where your things can fail.

But over here, there is prior tokens, which means infrastructure layer. Let's call this the LLM layer. 4 different transformer blocks all Usually the first content is the milestone. Why do we make this statement? You can complete your prefill phase but still fail to do your other things. Why does that happen? These are pure engineering. Pure engineering is you can have your proper startup but you cannot have your final output. Why possible? No, anytime it is possible. Why does that happen? Not related to LLM in this case, but people who are, who are software engineers, they can think a little bit here. Why your first question? You had your first token coming in.

Does it guarantee the entire Output. Let's say the answer should be check the payment service. It started check the correctly, but it did not eventually completed it. There can be a lot of problems from the engineering perspective also. Timeouts, network security. You can, you can think about hundreds of problems at least from the engineering perspective. Did I told one time where

we have a huge map and then I told that there is one security, district security, and in that there are multiple small topics. Go through that whenever you get chance. There you can, you can imagine a lot of such questions, but you need to ensure That you are having proper calculations of how much time it takes for the first token and explicitly defining which part takes what time, so that at some point of time you should be able to come and then you should be able to debug that behavior that something has gone terribly wrong.



vLLM: measure the first streamed text

```
import time
from openai import OpenAI

MODEL = "Qwen/Qwen2.5-7B-Instruct"
client = OpenAI(base_url="http://localhost:8000/v1", api_key="EMPTY")
messages = [
    {"role": "system", "content": "You are an AI SRE."},
    {"role": "user", "content": "Payments are failing. What logs do you
need?"},
]
start = time.perf_counter()
first = None
pieces = 0
stream = client.chat.completions.create(
    model=MODEL, messages=messages, max_tokens=300, stream=True
)
for chunk in stream:
    if chunk.choices and chunk.choices[0].delta.content:
        if first is None:
            first = time.perf_counter() - start
        pieces += 1
elapsed = time.perf_counter() - start
```

```
print("Client TTFT:", first)
if first is not None and elapsed > first:
    print("Streamed pieces/s:", pieces / (elapsed - first))
# Streamed pieces are not guaranteed to be single tokens.
```

vLLM: inspect server-side phase measurements

```
curl -s localhost:8000/metrics |
  grep -E "queue_time|prefill_time|decode_time|first_token" | grep -E
"_sum|_count"
```

Context pileup

But there is a specific point of view. Let’s say that your first call will have an X number of tokens. But your next, third, and fourth call substantially increased. Why did it happen? Let’s say that your AI SRE started with saying check the payment service output with some input. Next call it has 8,852 tokens. Then after that it has 10,298 tokens. Why that jump happened? And why do we need to accumulate? Right now what are we trying to do? We are trying to observe. This is an observability part a lot of times so that you can work upon fixing a lot of these things. For example, let’s say that you have a check the payment service.

Your model said check the payment service. Your agent program went ahead and given the logs to your agent. Now, what is the work of an agent? Work of an agent is to propose a patch. Now, in this case, it has the logs and then it has to also produce a longer decoding phase. That’s why it has become 8,852 tokens. Context pileup, correct. The later calls may increase the more and more the context. That’s why even your Claude or CoMS, that’s why we say, for example, your GPT or anything, they start to deperform pretty badly if the context gets increased, or they tend to also slower the things because the prefill phase increases.

What can we do? For example, let’s assume when we go to the design pattern— I’ll write this separately into the architecture— but when we go through, let’s say that you have a second output. It has 8. Let’s say that I’m at

call number 3. We will look at when we design this, when we put this kind of matrices, then we will put it like this. Oh, okay, can we do something here? Our context pileup has started to happen here. Can we do the summarization here using some small language model? It has happened over here that can we train some small language model for a specific task so that I can do the summarization over here with a highly specific thing which I want to do so that I can Decrease my delay.

Overall accuracy also. Increase my overall accuracy of my models and decrease the overall latency throughput also. The approach is that we should be able to put this out. That's why it eventually comes in observability part also, so that you can further come and debug that part. That's why you need to be able to come to and say, okay, what is happening at every single stage? What is pre-filled? What is being stored into my memory? Is there any way that I can reduce my working memory? And all that. Let's do a simple calculation over here. You have 1,024 input tokens and 32 output tokens.

Okay, you have 8192 tokens. And 32 output tokens, it will increase substantially again. Once again, it will increase. If I put the measurements, if you have 32 output tokens, which is fixed, your prefill will increase, your output will eventually decrease because your decode time. But if your output is eventually high, in this case, 128 output, in this case, your decoding will also increase. Ultimately, your point is to be able to come to the point of working around the trade-off is how do you decrease the prefill phase and further decrease the decoding phase. Most of the time we work towards decreasing the prefill phase.

A question is here. Prompt processing got faster. Lower client TTFD alone, server phase measurements under matched conditions, more output tokens per second. What has happened? You made a claim prompt processing got faster. What is server phase measurements? Your prefill went down and your decoding and so forth went down. It does not mean that your entire infrastructure has been correct. No. You are given a compute budget. In that budget, you cannot increase it. You need to separate that thing out and then you just have to focus on these 2 specific nuances. Which is prefill and decoding phase. Then only you can claim at least your prompt processing got faster. Then when you design POCs, you should be able to come and say,

okay, given this budget, I will remove that factor of what is compute, what is the memory bandwidth.

I have removed that factor, but I'll work on this particular phase. If you increase it, my prompt processing will be visibly faster. But if you keep that up, Just, I'll remove that factor and tell you that this is how I have improved the prefilling of the code fields given the compute constraints. Then when you can prove that if you give me enough bandwidth and enough compute, enough larger GPUs, I should be able to show you the substantial improvement. That's why we did in SLMs. In SLMs also, we say, first of all, let's get it trained on the 100 examples. If it, if it shows enough substantial behavior, we should be able to come and say, okay, now we can train it on 500 examples or 1,000 examples, then we can deploy more money.

Similarly, here we first of all prove it with prefill and decode phase.

What we did till now, nothing to make our inference faster, but only so that we can zoom in into the generation, what exactly has happened, so that we can know that which one, which one of our call is taking more time, which one of our call is taking less time, how should we improve it further. That's the next set of questions which we'll try to ask. We have not done any improvement to our inference service. The only improvement which we have done Call this as an observability of LLMs from a deeper perspective.

KV caching: why repeat the work?

That we were doing calculations. Attention, every sequence. In a sequence, there are multiple tokens. Every token was going through and parallelly to attention block, FFN block, 36 layers behind it. Then it was going to the next token. Next token was also doing something similar, multiplying with the attention block, multiplying with the FFN block. Multiplying finally the next tokens, next token. Basically all the tokens were behaving like that. All the 6 tokens were going through the entire. We were doing something like multiplication of token representations with keys, queries, and values. You have the sky is— for every token you have its own key, which means 32 ways of representing. This is what I do.

There can be multiple ways, 64, 32, whatever. Take an example. Sky is Everyone is having their own computations. But let's say that you are that you have that you calculated the sky. Okay, both of this. You come on is over here. Now you eventually have the next token. You calculated these sky. You calculate the keys values. The first sky you calculate the keys values for is. You calculated their keys. What is keys? Keys is like, this is what I represent, this is what I match with. And values are, this is what I actually represent. It suggests the next output, blue. Here you have the K4 and V4. When it suggests the next token, it is again going— because what is happening at every entire pass, what is happening, every token We are calculating their keys.

We're calculating their V's. We're calculating their Q's. We're calculating all the W's, wherever your W's are. In this case, we're calculating also the K. Whenever you have to predict the next token, now let's say after blue, we need to predict the next token. We will again go back and calculate K. Again, we'll go back, calculate sky. Again, we'll go back, calculate this. Again, we'll go back, calculate blue. Then to get the next tokens, then we'll keep on doing all of that. The question was, can we cache the calculation?

Q represents the current word that has to be computed. You have to compute the query because that represents the current token. You have to calculate the new queries also, keys also, values also. Let's say that I want to predict the next token after blue. Okay, let's say I want to predict the next token after blue. My attention block and my FFN block will go to B, will calculate the keys and values, then we'll take the query to calculate the keys and values. We'll calculate this sky again, keys and values. Then it will predict the blue. Now to get to the next blue, after the blue, again it will go and calculate the keys values.

Again it will go to calculate the keys and values for the sky as well. Again it will go to calculate the keys and values for the is as well. What is happening over here? You are literally, for every transformer block, what is happening is for every transformer block, you are going to recompute your K, Ks, Bs, and eventually your, the next tokens. What I'm saying is, ultimately, the question is to get to blue. If we have already calculated the sky and is, which of the earlier computations we can use and put that in our notebook so that we can reuse it for the next token? There is a high possibility that we are saying let's,

let’s just go ahead and use the keys and values because we have already calculated it.

If I’m at blue, I will calculate its keys, queries, and values, all 3 of it. But because I have already calculated the keys and values for the is, why we don’t do query. Query is more about here’s what I match with. Here’s what my question is. Querying is of your new token. You want to go ahead, you are not multiplying to get to the new token, you are not multiplying the query of this with something, with the query of is or query of sky and query of be. You are taking the new token and then you’re taking this token and then you’re multiplying the query of it with the keys of is, with the keys of sky, with the keys of be.

Once you have the Q multiplied by K for all the 3 previous tokens, You are going to combine that with V, the softmax. Basically, you are first of all comparing the weight of blue with all the keys prior. Then you’re combining the effect of it. Then when you are getting the final H, the H is what? H is basically your final computation, the representation of the context. Then it goes to the FFN. That FFN is taking it to 192 and then come pushing it back to 64. Here, when you were predicting is, let’s forget the you were predicting blue. When you were here, you already calculated its is keys and values.

Why to recalculate it? Because you already calculated its keys and values, you don’t want to recalculate it. Let’s just reuse it rather than Blue already has its query. It means to go to multiply with the keys rather than recalculating the keys again by multiplying this token representation multiplied with the, WK. You get the keys. You have already done that part. You want to reuse that thing again, and then you want to reuse that thing’s size again in the attention block. Because what we are doing, we are multiplying the query with the keys to compare. Blue is being multiplied with is sky keys and the keys, but those keys were already computed.

Why not store those things so that further ahead we can reuse the computations?

Why is and sky and bee keys and values won’t change? You know why? Because it is not dependent upon blue. It is also, if you think logically, its keys and values will not tend to change because is not dependent upon blue, but

blue is dependent upon is, sky, and bee to be able to understand the context. It can reuse their keys and stuff, but it cannot reuse Blue. Technically, the context will not change because it does not know the future. But Blue already knows whatever has happened, so it can use what they represent and what they actually contain. That won't change. For example, let's take a Tinder example, our original example.

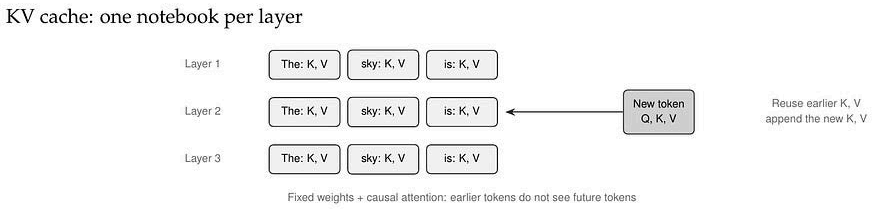
Your people which you have already viewed, does their actual personality which you already know what they represent in the profile and what they actually had in the profile, will it change? No, we have already known. But the future girls, you might say, okay, they might change, they might be able to have something different opinion. But at least what they know, you should be able to know, okay, they won't change what they represent and what they actually are. I can reuse whatever I can from them side so that I can get to my next possible prospect. Ultimately, I am saying over here is you, if you have— if you understand this more logically over here also, Because your weights are fixed, your keys and values are not going to change up there.

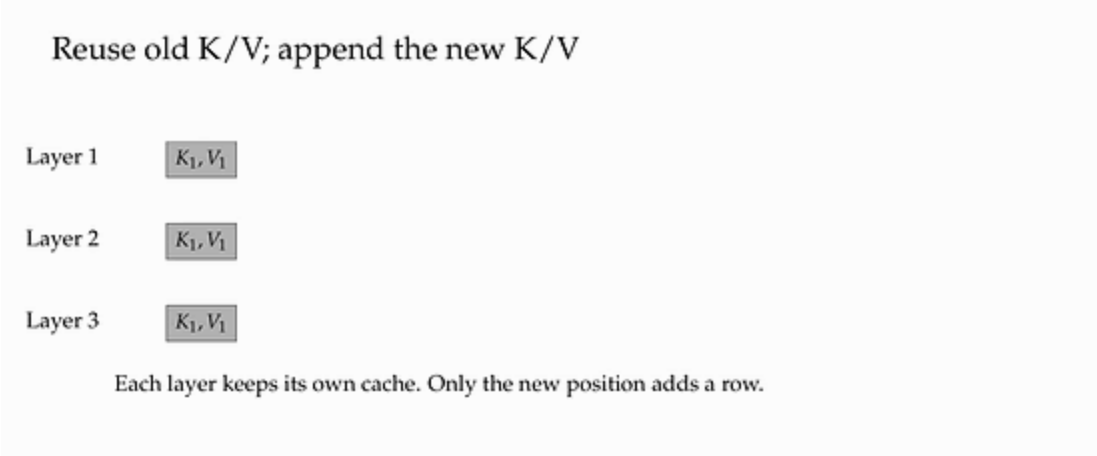
Why am I saying it is appending a token? Because it is causal. Why am I saying causal relation? You will, it will make much more sense. No. Yes, correct. Perfect. Now you're saying blue is dependent upon these 3 to understand the context. Is eventually dependent upon these 2 to understand the context. Correct. Now imagine, because it is not dependent upon future, these 2 have already been constructed. It is of no possibility that the causal mass causal relation will also change over here. The causality will say, oh, you know what, you already have already constructed my DNS type. Keys. It won't be changing because I don't have the access to the future information.

I have already calculated what I knew. Until and unless you change my how I should look at things, I won't change. Even blue over here, do you think blue is dependent of— is having access to new information? As soon as it has access to new information It will start to change because when you have access to new information, your keys and the values will change. But your transformer rule is only to not allow to see future information because otherwise the training problem is not there. Next token prediction, you only have access to the previous information, but because previous we have already calculated already that the sky is has already been calculated, We will not really see the change in the keys and the values up there.

When it was being calculated. But as soon as you say, okay, you have access to the future information, it will start to change because you also look at the future keys and values then. But that's not how training problem also occurs. The next token prediction does not work like that. You compute for the blue in this case. You already, because your weights are there, you already compute the new position, also computes its query. The blue will compute its query. And as soon as a new query comes in, you compare with the keys of all the previous tokens. Then you get the attention that how much my blue matters to other tokens available.

Then you combine that with values, which is what your attention equation also is, the context for this sequence. In this case, In most of the equation, vLLM already handles the KV caching up there into your— if you enable it, which is KV caching, then ultimately it eventually handles it. But what is happening internally, it is reusing the computation.





vLLM: ordinary KV caching without prefix reuse

```
from vllm import LLM, SamplingParams

MODEL = "Qwen/Qwen2.5-1.5B-Instruct"
```

```
llm = LLM(
    model=MODEL,
    enable_prefix_caching=False,
    kv_cache_dtype="auto",
    max_model_len=4096,
)
output = llm.generate("The sky is", SamplingParams(max_tokens=3))
print(output[0].outputs[0].text)
# Ordinary KV caching is still managed by vLLM.
# enable_prefix_caching=False disables cross-request prefix reuse.
```

One notebook per layer

If you have 36 layers, right, one block Is one block. Okay, six blocks. Do you think that we can reuse the first block piece in the second Why? Because the first block key has thirty-two different recipes of looking at these, but the second block also has different ways. Exactly, thirty-two different ways of looking at things. What I'm ultimately telling, I am telling that we will not be mainly looking at you. We can only cache the sky is in one. Whatever the keys and values into just one layer. Other layers, there is a separate cache. Caches will be there.

Right now, another question comes. To the lot of people. They say that, I, if I am storing my— if I have, let's say, 2 attention heads, do you think the keys and the values will be different In one block you have 2 attention heads, let's assume. One attention block has 2 heads. We have 2 different lenses. The keys which you'll calculate in one attention head will be different than the keys in the second attention head. Will be different in the third attention head, will be different in the fourth attention head. They will be different. All the keys, because the weights are different. But attention, what we say, we can divide the attention from 2 lenses, then we have a separate way to combine it.

Anytime attention head be changed, the weights also change over there. Because you are not done. You're not. You're just splitting that, and then you're learning a different lens if you have two different heads. Similarly, exactly. Similarly, what t to say is you can have two different KV one. You can have two different KV heads. If you have thirty two different. Heads, then you have 32 different KV heads, which are caches in general. In grouped query

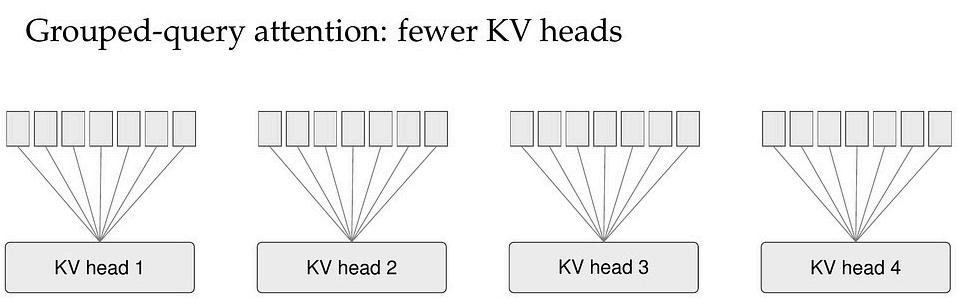
attention, you'll have the GQA. You will be having a— if you have 32, then only because when you query, when you group the queries, you will be having just 4 KV heads. You study about grouped query attention.

It's pretty simple to understand. You're just querying the keys to be able to make it more queries to make it more efficient.

If you talk about inference, your weights do not change. You have 36 layers. Correct, 36 different layers. Every layer will have their own weights, so every layer will have its own cached KVs for that sequence. Second layer will have its own cached KVs, third layer will have its own. It cannot reshare KVs between layers. If you think logically also, imagine keys and values are what? Keys are 32 different— I'm taking example— 32 different recipes in first layer. Values are 32 different recipes of— this is what I actually contain. Second layer, it will be having a different length. D will have— in first, it will have its own keys and the values in the first layer.

32 different recipes of looking at. Second layer will be different because weights are different. Third layer will be different. Fourth layer will be different. Fifth layer will be different. All the way down. You cannot reshare your keys between them, but you can actually retain that. Whenever a new token we want to predict, we can just reuse from that layer, the first layer. You anyway have to calculate 36 different— you have to multiply queries of keys 36 different times up there. You don't have to recalculate keys and values again. You're just going to use whatever you have used. It has to compute all the input to get the first token. And once compute is done, it is picking from the cache whatever computations which has already been completed.

You do one thing. Let's imagine that Yes, cache with respect to each layer. Anytime your weights will change, even with respect to your heads. For example, there are 2, 2 different heads. They will change because there are 2 different query keys and values which are being constructed. Later it is being combined, that's a separate topic altogether. But at least that is something, we are getting different keys.



Qwen2.5-7B: 28 query heads, 4 KV heads, 7 query heads per KV head.

Model-specific example from the book; the transcript also uses a separate 32-head example.

How much memory does KV caching take?

We have the cat sat on the mat. Every tokens get an address. Every token from the embedding table, we draw an address. In this example, every token can be represented in 4,096 different representations. Instead of 64 here, the shape of every dimension says— or of every token is 4,096 dimensions. Now in this example, our attention layer has 32 heads. And each head has around 128 dimensions. Over here, if even you multiply, you'll come— when you combine all of this, you'll go back to your 4,096 dimensions. For every attention head— why do we have multiple attention heads? We have to look at multiple lenses when we concatenate them.

For every attention head, the size would be 32 for 1 token, 32 rows and 128 columns. Right, because we have divided, we have multiple 32 attention heads. We, for 4,096 dimensions, was we took, we took that and said, okay, we'll be having 32 attention heads. For every attention head, we'll be having 128 dimensions. When you multiply 32, that is coming back to 40,096 when we combine all of that. But we expanded it so that we can look into multiple lenses. Is if I want to, is this the correct shape? 32 for one token, 32 different ways of looking at things and 128 dimensions for one attention head. Second will have its own 32, 128.

Third will have 32, 128.

Over here, can I say for all my 6 tokens, my shape will be 6, 32, 128 for all my 6 tokens? The shape of my keys can be 6 multiplied by 32 multiplied by 128, the shape of my keys. The value of my key, the shape of— Yeah, value of my would be also same. If I want to know the shape of both of it, can I do 2

multiplied by 6 multiplied by 32 multiplied by 128? It is one format to store. We'll, we'll know, we'll get in detail about all these formats in GPU programming. What do you know about this format, BF16?

What you're saying is, you, when you multiply, you get in bytes because we are following a 2 bytes per number. Over here we are following a BF16 format. There are multiple 4-bit precision weights and there are multiple formats. But right now we are following a 2 bytes per number BF16. When you multiply, you will be having 98,304 bytes. If you convert that into KIBs, which is another format to represent the numbers, which is basically to make sure that it is readable enough in my opinion, KIB, and then 96 KIB, 1 layer for 6 tokens. For 32 layers for 6 tokens, we have 3 MiB per 6 tokens. And if you really want to calculate per token, so 3 MiB, which is your 3 MBs divided by 6 tokens, that is your 0.5 MB per token, which will be the space it will take to store your keys and values into your memory bandwidth.

What is ultimately, if you say, let's say that your input prompt has around 1,024 tokens. That is going to be 0.5 multiplied by 1024. That is the space which will take MBs, that will take, and then you convert that to GBs at that time.

I'll show you a worked example here also. Let's say that you have 2 layers into your, transformer. You have 1 KV head, which is basically you want to store all the KVs. You have 4 coordinates per vector, which means every word can be represented 4 different ways. You're using a BF16 format. In this case, you're going to take your KVs, multiply with the number of layers, multiply with the number of KVs you're having. If you're using a group query, that is going to be much less. If it's going to be multi-standard, then for every keys and values is getting stored, there's 1 KV head, multiply by coordinates, multiply with the total number of bytes.

Here we are using 2 bytes per stored number. You're ultimately going to get 32 bytes per token in this architecture. Similarly, for a different architecture, let's say that you have a Twin models right now which I use. You have 28 layers, you have 4 KV heads, which means you're using a group query attention for every particular, per head. In that case, we have per head, we are using 128 dimensions bytes per BF16 number is 2, which means 2 bytes we're using, to— this is a way our computer stores any numbers or any

information which you're going to store here, the total space we are going to have is 56Ki, which is 57,344 bytes for this example which you have taken.

If you convert that to MBs— sorry, bytes to KiBs, which is per token— and for 1024, you just need to multiply this and then you're going to 56MB. This is for just this layer, but larger models have more than 36 layers and more than that. They have much more KV heads. Their coordinates per head will be much larger than 128. And then you want to calc— then the— for just the input tokens, that's the space which is required just for your working memory to be stored.

Your file size is 3 MBs, 4 MBs, 10 MBs. Similarly, some are in GBs. There are the ways to store information. Similarly, to store your weights. They are also stored in a particular size. What is the size of a— what is the space needed to store per token is 56,000. KIB in this case. Similarly, if you convert that for storing your entire tokens, 1024, you're going to need around 56 MBs. And that is for just one sequence which you're going to send. You are not going to just send one sequence, you're going to have more and more input tokens. They will require more and more bandwidth to be needed. The layers will also increase over here, layers, the KV heads, the coordinates per head, etc., etc.

KV memory: one token, step by step

Step	Calculation	Bytes
One key vector	128 numbers x 2 bytes	256
Key and value	256 x 2	512
Four KV heads	512 x 4	2,048
All 28 layers	2,048 x 28	57,344
1,024 tokens	57,344 x 1,024	56 MiB

BF16; 28 layers; 4 KV heads; 128 coordinates per head.

KV memory: calculate the three transcript examples

```
def kv_bytes(tokens, layers, kv_heads, head_dim, bytes_per_number):
    return 2 * tokens * layers * kv_heads * head_dim * bytes_per_number

# Two stores: keys and values.
print(kv_bytes(1, 2, 1, 4, 2))         # 32 bytes per token
```

```
print(kv_bytes(6, 32, 32, 128, 2))     # 3,145,728 bytes = 3 MiB
print(kv_bytes(1024, 28, 4, 128, 2))   # 58,720,256 bytes = 56 MiB
```

GPU allocation is not live KV occupancy

Are you saving KVs real time? For example, the new tokens are coming, the new tokens are being generated, the new tokens you're processing, you're doing it real time. You're doing inferencing real time. You're doing in real-time inferencing. Even your keys and values are being— for the new tokens are being generated. Those are— those also has to be stored. GPU allocation is not live KV occupancy. You can estimate that my maximum token into working session will be this much.

Similarly, when we design our agent, which is AI SRE, we need to ensure that when we— when a user, when it eventually goes ahead, looks at the incident, then how much of the memory bandwidth we can give per request. After that, it should start to work around either deleting information or making entirely a new session or caching, what it has to do, we'll figure it out. But that is a strategical thing that you have to decide, that when we come here, we will calculate that we are using Claude. If you are using Claude, then you don't have to worry too much because Claude will do the work. But if you are using open source models, then you have to make sure of the models.

But if you are talking about Claude APIs and all, then not much of a GPU allocation because your enemy is going to pay per API. But if you're using open source models, when you're implementing an inference service, you have to ensure that your GPU allocation, specifically with respect to the memories and bandwidth, are taken care of. GPU allocation is not live AV occupancy. Does Why? Because KVs are again— KVs are not fixed. The new tokens, you're going to keep on generating new and new tokens. The keys and values going to retain. Thus you have to save your keys and values. You save your keys and values, then your runtime will— your overall memory bandwidth will also increase.

But you have to intentionally open that layer. When I say intentionally open that layer, you have to— why you're opening prefill and decode layer, why you're measuring latency, so that you can figure out what can be the strategy.

I— whether you can summarize your input tokens, keep on summarizing your input tokens, but what is the trade-off in summarizing input tokens? What is the trade-off if we keep on summarizing input tokens? You will miss important things if you keep on summarizing. You will develop strategies on top of it when you open that layer. That's why the first entire time it's talked about pre-code and decode and the KV caching is open that layer, then later build on top of your strategies, agent compositions, whether to summarize, whether to remove Whether to decay information, that is again your strategic engineering decisions.

Similarly, when you work with, let's say, Claude, you will definitely work with Claude. You will want it to expand these 2 things, phases, and then independently work both of them, on both of them. Then you will define, okay, now if we fill there, this is what is happening. After every token tool call, my request, my prompt size are doubling. Exponentially. What should we do there? When you open that thing up, that time you'll understand. Ah, here something is wrong. My prompt size is doubling it.

vLLM: set the memory fraction and context limit

```
vllm serve Qwen/Qwen2.5-7B-Instruct \
    --gpu-memory-utilization 0.90 --max-model-len 16384
```

Prefix caching: the same start across calls

But now imagine I will tell you one thing. Let's say you have 5 model calls, and over here I want you to understand the model calls. Call number 1 was usually you have seen system prompts. System prompts is how much? What is you are an AI SRE, blah. This is what you're allowed to do. That is what you're allowed to do. Hundreds of things as instructions. You have the system prompt, then system prompt. After that, you attach the new, which is tool results. For example, in call number 1, you said you are an AI SRE, these are the available tools, and this is the what we are building. Then output.

Then you ask, can you tell me what is wrong? Which goes to the model. Model says check the payment logs. In the next call, you're using the same thing. You are an AI SRE. These are the available tools. This is the thing

which you are building. This is my previous result. Then check the payment log comes in over here. The payment logs get attached. What is changing? Instructions and tools remains the same. What is changing is what you're asking every time the context fill-up is happening. Every time you are having the new context getting added from the previous call. Why am I saying from the context fill-up is happening is in the first you say this is what, please tell me what is wrong in the call number 1.

You are an AISRE. This is what has happened till now. In call number 2, you have attached the new thing, which means here is the payment logs because the model says give me the payment logs. In call number 3, model said apply this. Instructions remains the same over there in multiple calls. The start of the prompt You are an AI SRE. These are your available tools. They all remain the same for every call. Usually that is the case. What is changing? Your tool results from the previous call, your model which has, which is produced, or logs, the new values which is getting changed at every single call. That's why this gray one says, gray value is saying that, hey, I remain the same at every call.

What is changing is basically the output from the previous one. My prompt is same. You are an AI SRE. This is, this is, these are the available tools. You are an AI SRE. These are the available tools. This is the context. This is the access to the, this is the environment. Those prompts do not really change. What is changing? Your ultimate byte values. In this way, you have some output from your models here or logs attached. In this way, something is different over here. In this way, you have a call, maybe the model logs or anything of that kind, which is in call number 3, it is increasing. Do you think because the start of my prompt is almost similar, the start of my prompt is almost similar, is there any way I can cache?

I can just cache the one which is similar. But if, for example, my URN, AI SRE, here my available tools, I can save those as well so that in the newer calls, if I identify the similar language from the start, I can just reuse that rather than calculating, recalculating Ks and Bs for that as well.

See, again, I'll try to explain. Yeah, is prefix caching— this is one of the techniques. What are you going to do? What I— in every prompt which you send to the model, because you want it to know at every single time what is

my available tool, so model has to process every single thing. Rather than in one call, KV caching is used in just one call. KV caching, which we introduced, was being used in just one call. If we go through this, we call this openai.generate, then we, if we re-give the same things, it is not going to use the KV cache from the model call number 1. Because when you call an API, whatever happens is over there.

What we are saying is the second API call, I want to use your instructions, which has already, if the instructions in the call number 1 and call number 2 is same, Let's reuse those. Let's cache those things and let's reuse, which is KVs or KVs from this for the instructions and tools, which is shared one because your instructions and tools are similar.

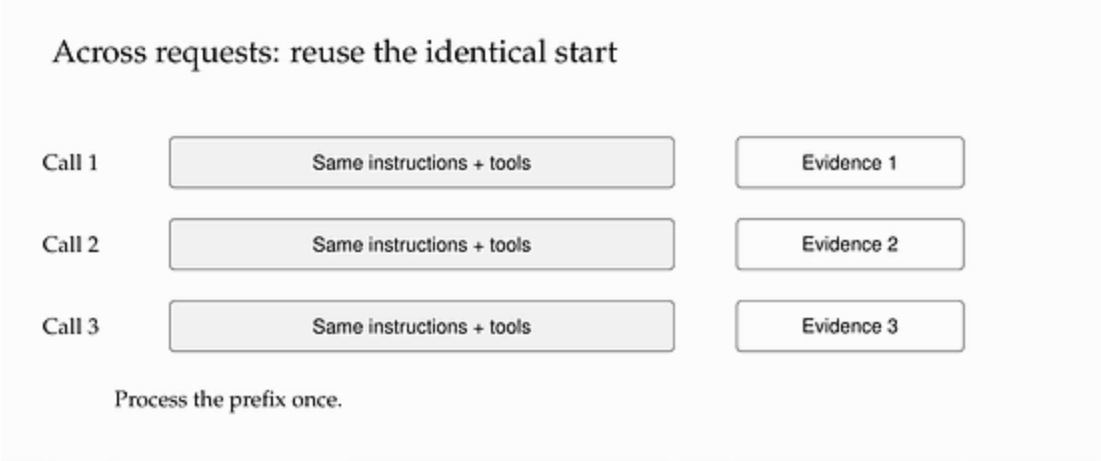
What is my instructions? Let's understand. Just I will give you so that for the people who has not understood, I'll just give a small example. That the your instructions can be your system prompt or every single time your instructions to your agent whenever you're building it can be you are an AI SRE it can be What should be your way? What should be your way to investigate? Then second can be what tools you are allowed to use, or what do we have, the databases which you're allowed to use, etc. Now this block remains the same in every call. In one prompt, in one call, you can say new results. Now, new results is basically whenever this prompt goes to your model, your model can say that, okay, you're an AI SRE model, say check the payments logs.

The next time, this will remain the same. You are an AI SRE. These is the instructions which you can use. This is the tools which you can use. Now this is the result. Red ones remains the same in every call, in every harness calls which the model is doing. What changes? Your new results. What is really happening is your tokens basically same. Let's say that you have in every call your instructions contains about 3,000 tokens. Okay, same 3,000 tokens in call 1, 3K tokens in call 2, 3K tokens in call 3, 3K tokens in call 4, all the way around to 3K tokens in call 6.

They all contain the same instructions. If we do not use prefix caching— let's assume if we do not use the prefix caching, so there are around 18,000 tokens which we recalculated. Let's assume 1 token is following, let's say, 16 bits. Let's say 2 bytes. 1 token takes about 2 bytes. 18,000 tokens multiplied by 2, that is the amount of storage which you want to do. That is the amount

of storage in the memory which you want to do. If you just cache it off, you're going to store much more lesser. I'll talk about the memory in a bit. I just want you to imagine what is this length. You want to reprocess this 18,000 tokens again and again.

The same tokens, 3,000 tokens, 3,000 tokens, 3,000 tokens, 3,000. What is ideal behavior should ? Let's first of all, I will do my first call. I will process the instruction, but this time I will enable the prefix caching so that the new instructions I can just use, reuse the computations which were done in the previous one, just keys and values. I'll complete once and I will reuse the cached computations. Now this time, just because I have just done the 3,000 tokens computation. If you minus 18,000 from minus 3,000, you saved around 15,000 words of repeated prefix which you avoided.



vLLM: enable prefix caching

```
vllm serve Qwen/Qwen2.5-7B-Instruct --enable-prefix-caching
```

vLLM: count the cached prefix tokens

```
from vllm import LLM, SamplingParams

MODEL = "Qwen/Qwen2.5-1.5B-Instruct"
llm = LLM(model=MODEL, enable_prefix_caching=True)
prefix = "You are an AI SRE. Inspect payment logs before proposing a
repair.\n" * 20
settings = SamplingParams(temperature=0, max_tokens=80)
```

```
llm.generate(prefix + "What logs do you need?", settings)
result = llm.generate(prefix + "What evidence is still unknown?",
settings)[0]
print(result.num_cached_tokens)
```

Why prompt order matters

Now think about it. If I add a different in the instructions in starting, I add the timestamp, which means right now it is happening 3:35 PM. In call number 2, I add 3:36 PM. In call number 3, I add another timestamp. Do you think caching will work there also? At the very start, I add the timestamp at the very start. Now every time my calls will look different because the starting is not same. We cannot reuse whatever we have understood, what was our instructions and what were our available tools. If we add this timestamp, The vLLM has introduced all these things only. Eventually they will say, hey, all these calls will look new because the timestamps are added ahead.

What you need to do, you need to add the timestamps after it, after your similar instructions, so that my previous starting could be reused across the calls. To enable it, vLLM actually gives a very nice and simplistic view. You can just call the vLLM and enable prefix caching.

The only thing which it cannot use is following. If it starts by timestamp. If the starting is different, then it cannot. That’s why I told over here is if the bad order— if the order is correct, if call 1 has this much instruction similar, if call 2 has this much instruction similar, so it can reuse this much instructions only. Anytime a new token is introduced which is similar— which is different from the call number 1, Then it will evict from there. Then it will start to process. That’s why if you have an order like you first of all put the timestamp and timestamp will be different in every call. What happens if the timestamps are different in every call?

The next set of things eventually goes to it will be reprocessed even if the instructions are same. It does not matter if the instructions are different in starting. It will re it will reprocess those things. But whatever if call three. Has this much instruction similar? Sure, because I have cache which matches from the starting to this hashing technique. It is going to reuse. But if the starting itself is wrong, different, not wrong, different. Let’s say in this case

timestamp, then it will not process. Then it will reprocess even though just after it is a similar tokens because the context can change. Why? Why is this the case? Imagine I put a different wording over here.

Why it is not possible to cache? Because we do not know what is what is it starting. We do not know. That can change my entire— maybe do not follow can be one of the token, but after it is my similar instructions Good idea. No, nice. But generally speaking, imagine I put do not allow, then the instructions and tools will be, will be correct. Obviously not. But caching can be used there because they use the same words, but it cannot be possible. But the— I — but the new tokens, but the starting words say do not allow, then no matter what comes outside of I should not, I should not have been allowed.

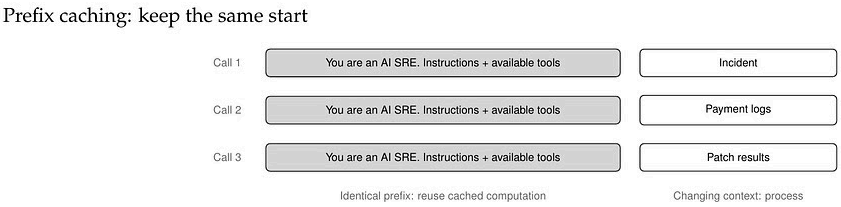
That's why the order matters over here a lot. If your order matches with the call number 1 and other calls, then it can reuse. But if it does not, it will not even— if it is any time difference, then it will not use, and even the keys and the values will be very different because the context is different at that time.

When I— if the prefix caching is off, my time to first token is 1 second. Time to write the answer is 2 seconds. When prefix caching is on, then my time to first open is 0.15 seconds. But when my prefix caching, when I put the timestamp at the top, that is 1 second. Why? Why there was 1 second if I put, if I change the timestamps? Why that was the case? Because I had my 5 calls, I just put the timestamp in starting. Why it, but the rest of the tokens were same, instructions were same. Caching did not work. The bad order, the problem of bad order. See, the reason why I'm telling this, why I'm emphasizing you on bad order, whenever you create System prompts.

You have to make sure this thing in mind. If you want to, you see, people will say, okay, use caching. But cache, for that to work, your this strategy of instructions and tools to be listed in a way, it should not be coming out of the mind. Like anything, you should have a strategy. Okay, my instructions and prompts to be in a way so that we can use caching. For that to happen, I should have it in the right order. That is one of the strategies, technique itself. To develop this, to have caching work, I need to make sure my orders are correct.

Okay. See here also, I have written clear stuff so that people would know. You need to decide what is the— that's why this is working with Claude, is you need to decide the order of prompt. Which is what is fixed, because your tools can change. Your instructions can change. How do you ingest those things? What will change? Which will come? Basically, what will change? Static things and what can change. Rest of the things can eventually go. For example, calls 2 to 6 show a high hit rate and much lower time to the first token. What it says eventually is if your— if let's say your caching did not work, your automatic debugging should be, hey, let me see if the orders are right.

That is the biggest problem. Even today, whenever I develop anything, if my orders are not right, and most of the time my orders are not right, Because my tools' literal name can change. In one prompt, something changed. In one model call, something changed. Whenever I fetch the tools, maybe the provider has changed the tools, tool names. That Claude will not get— no, whatever it has, whatever you have saved. But maybe there's a layer to standardize the tool names and then come inside it whenever we fetch the tool names from anywhere.



Prefix order: stable instructions, changing evidence

```
# Application functions/constants supplied by the harness:
# normalise(tool), SRE_INSTRUCTIONS
def build_messages(tools, alert, evidence, now):
    # stable parts first, in a fixed order
    tools = sorted(normalise(t) for t in tools)
    system = SRE_INSTRUCTIONS + "\n\nTools:\n" + "\n".join(tools)
    # changing parts last, the time at the very end
    parts = [f"Alert: {alert}", *evidence, f"Time: {now}"]
    user = "\n\n".join(parts)
    return [{"role": "system", "content": system},
            {"role": "user", "content": user}]
```

vLLM: inspect prefix-cache metrics

```
curl -s localhost:8000/metrics | grep prefix_cache
```

Repetition penalty

Another problem is what happens with greedy decoding whenever it says next check the payment logs, next check the payment logs, next check the payment logs. It keeps on selecting the next token. Because it has already seen, it is going to keep on recurring. It has happened that you might have also seen in initial days of GPT, especially in pre-training, it used to repeat the same words again and again. Even today, forget about that. Even today, what used to do, it used to also generate the verbose thing, very verbose language. You try GPT once, it will give you 1,000-page answers. On— you just ask, hey, can you tell me how to fix a small thing?

Imagine that you are walking down a baking path and then you take this. Oh my God, it will give you hundreds of things and it will repeat the same thing again and again at times, or it will use the same words.

Let’s say that your model said, next, check the payment logs. It keeps on generating: next, check the payment logs. It used to repeat its own words. Here your greedy decoding failed. We have told it to always choose the highest probability. In this case, the highest probability was always next, check the payment logs. We introduced another way. You have already used

this word several times, consider another word if it fits. We are giving a repetition penalty. Just like temperature, we were putting that parameter in our calculation when converting in our softmax. Here, let's say our logit is z, we can give a penalty to it, r. When r is set to 1.0, then no penalty.

You can repeat the words. If r is set to 1.1, we are saying mild penalty. Do not repeat too many words. Ultimately, a word which you have already used gets its score pushed down a little.

Z is your logit. Once you come, so you have 150,000 logits, no? Then it goes to the softmax. Over here, a word which I've already used gets pushed down a little because of this penalty. Now, if we understand what— and the code in general, temperature, let's say we have made it to 0, which means greedy decoding. Here we are using greedy decoding. But greedy decoding, you're saying let's put a mild penalty so that it does not repeat the same mistake. Sorry, same tokens. Why did we do this? Because of one simple reason, so that it does not repeat the same word.

We said, okay, I don't like such trade-offs, but can we create some way that can give us another words but not the same words? Because greedy decoding was having a problem producing the same words. Then people said, okay, we'll introduce a repetition penalty. You can use greedy decoding with repetition penalty so that your new words that were repeated already will not be repeated. Based on the values which you choose. Usually the industry standard is 1.1. If you use greedy decoding, depending upon your— things. But yeah, how will we ensure that we are choosing the right values of temperature? Evals, evaluation datasets, which you create on by hand to be able to ensure that how much creative it is getting at multiple use cases from a 360-degree angle.

That's why evaluations in agents are still an unsolved problem or a bigger problem.

Beam search: alternative paths

Assume that you have to complete this word. The payment incident was caused by dot. We can have multiple continuations.

What happens is beam search says that let me keep the multiple candidates. Let's not pick one candidate, let's pick the multiple candidates for a continuation. It obviously costs decoding time, but here what we are saying, we are saying one candidate can be this, one candidate can be this, and one candidate can be this to select from. Then what you're doing, then you eventually say, okay, now I will select the one, let's say out of these 2, I will select the, we can have several paths, but I will select any one of it. I'll just give an example up here. Let's say that you continue the, what was the next token?

Next token was payment and config and network. 3 things. To continue the sequence, I will select if your beam width is— if the parameter of width equals to 2, you want to select the 2 tokens to continue with, 2 best path. Then you select the payment flag on, then config flag on. And then you can have the another timeout, which is payment timeout. And then for the config, the config typo. There were 4 possible sequences out of which the most probability thing were payment flag on and config flag again on. Out of this, what you want to do, you want to select whoever has the highest probability over here.

Out of which you say, okay, I want to select the best. I'll select the candidates who can be possibly my best words. You maybe select the top 2 probabilities, and then you continue. In the continuation, you wanted to generate 2 answers because your width is equal to 2. You say payment flag on and payment timeout. Similarly over here, then you similarly over here out of this 2. Your top probability was this and config flag was this. Then you compare the probability between both of them. You want to select with whoever has the highest probability. The reason why we do that is most probably for a simple answer, is basically you want it to generate accurate short answers.

You explore alternative paths by selecting n number of candidates. By selecting n number of candidates here, because we have said possible number of, width equals to 2, we want it to be generating 2 possible sequences. For every such continuation.

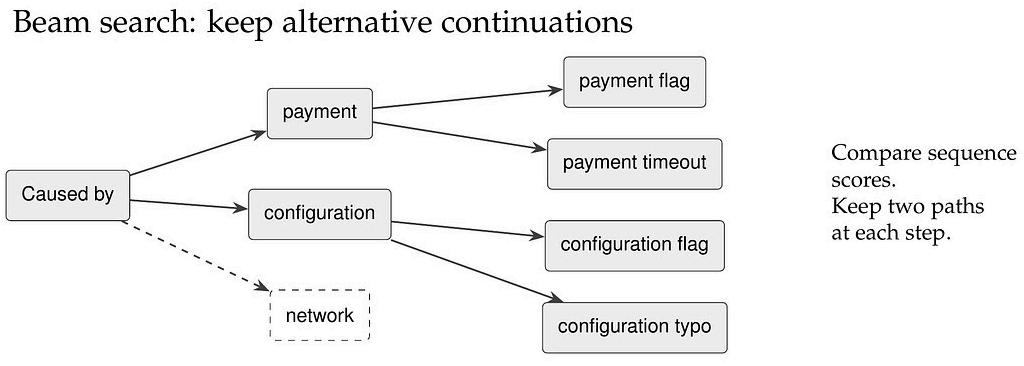
The reason why we should do it's not very famous to do it, but what we are saying is because it involves your decoding time increases because you're doing multiple comparisons here. You wanted to select the one which has the highest probability, but also compare the multiple stores here because what

happens Your best next token, understand like this. Your best next token will not lead to the best whole answer. Instead, your next word, let's say if you just go ahead with payment, and then it will say payment flag. Then it will say payment type, etc. Basically, you just say payment flag. The flag is the next token. It will keep continuing because it has chosen payment.

But what if the best answer would have been, let's say, if you choose a config here and then you go ahead with this, but this time, but here the whole answer would have been great if it was this, but the token was not predicted nicely. What we say is you generate, you go ahead with the 3 candidates, whatever n candidates. And then you generate 2 possible sequences, or you can make it even more possible paths. Here, maximum 2. You're saying 2 possible paths, and then compare the probability for path. You select what is the top probability, and then you compare both of them. Why we are doing this, again I will emphasize, is your next token is not always the final answer to compare.

What we do is we take both of the paths, and then we try to compare them. Beam search.

But here, at least in inference, you can just use beam search to be able to. But here what we do is we generally do this for shorter answers. Why shorter answers? Because that will increase our decoding time. Because we have to compare also and then finally select one. And also overall time length as well, time and cost. We generally do this for stricter answers and simply accurate answers, and where our answers are much shorter also. Team search in general, it is listed over here. It is also useful for the short and exact outputs which you're looking at.



Illustrative beam width: 2. Candidate labels are examples, not diagnoses.

vLLM: compare two beams

```
from vllm import LLM
llm = LLM(model="Qwen/Qwen2.5-1.5B-Instruct")
prompt = "The payment incident was caused by"
from vllm.sampling_params import BeamSearchParams
outs = llm.beam_search([{"prompt": prompt}],
                       BeamSearchParams(beam_width=2, max_tokens=20))
```

Speculative decoding: a junior and a senior

What you do, you ask your model to generate the, multiple tokens, which means you ask your model to say the payment flag was Which is the next set of tokens. You ask your draft model, and then what you do, you have your big model checking whether each one is correct or not, which means, yes, is this a correct one? Is this not the correct one? The junior model is proposing the next 4 tokens. Junior— why junior? Because you'll be having a— when I say junior model, Why do we differentiate between what is junior model and senior model? This will have more parameters. This will have fewer parameters. This will have more parameter.

What happens because it has less parameters? It can generate multiple tokens faster. The prefill phase can be faster because it has less, parameters. But now your bigger model does not have to look at the entire prompt. It has to just look at what it has generated. The prefill phase is only on 4 tokens. It can tell that which tokens were correct or not, The was correct. The big model said payments was correct. Flag was correct. Was not correct. In this case, you put is, which the bigger model is saying.

Speculative decoding is more about how to make the models faster to generate. What happens? Let's say that you have your big instructions. Instructions: you R and AI, SRE, blah. Instead of giving it to the higher parametric model, which will take a lot of time, you give it to a model with fewer parameters. The prefill phase is much faster. You generate 4 tokens here using a model with fewer parameters, and then you have a powerful model just checking these 4 tokens. It happens, Rohit. Speculative decoding,

it happens. You will lose the context. You lose the context for sure, no question in that. This is one of the methods to make things faster.

Instead of— so what people do is You have maybe model things, model like GPT Sol and powerful models, but they must be using Luna to be able to quickly do the things and then the powerful model quickly rechecking. Think like that. Usually it works also most of the time, which means you wanted to just correct it if anything goes wrong. For example, why current are able to offer Sol and such things at a cheaper price, Sol Lite. For one of that same reason, is lower parameters model to do the prefill phase faster to generate something, and a higher model to just, instead of processing the whole tokens and the context, just process what was generated.

Or they may offer some context as well if you think that some context is being lost to be able to process. We don't send our original instructions to powerful models. We just send original instructions, logs, and to lower models, and whatever output we get from that is given to the powerful models. Powerful model, instead of processing the entire 8,000 tokens, it processes only with little bit of context, if at all you need. It's like junior who will do all the pre-fill phase of a longer prompt and senior who will just come and check your work rather than doing all your checking what you did, how you did it, etc. It does not even go to the pre-fill phase.

It does pre-fill phase, but only in these 4 tokens. That's why you get a powerfulness of a longer model. But what is the trade-off? Trade-off is Context, hallucination, etc., etc. But what does it help in decoding?

What if the senior model does not agree with generator? It replaces. No, senior model will not do the prefill phase. We can have that engineering strategy, that if my model does not agrees with more than two tokens and changes it, we can route it do it. Do the prefill phase. That is definitely possible engineering-wise. You just have to create your harness in that manner. It will just change it. But if more than 2 tokens, let's say only 4 tokens, and if more than it disagrees with more than 50% of the tokens, go ahead, read the prefill phase, definitely. Or we may have a shorter context for it, a summarized context from the doc.

Summarize context from the draft model. Multiple things can happen. Yeah, see, I can think of multiple things. I can think of draft model, bigger model changes the things if only one or 2 tokens are there. But if it is a lot of tokens, then what it will do, the model, draft model can give a summary and then ask it to generate the new ones. If the draft model is not good at summary, then go ahead and look at the entire things. Those kind of things can be done. Multiple ways to solve this problem.

But you're absolutely correct that actually save time is heavily dependent upon your evals. You truly test it. For example, here I have done it. Is I'm checking it every single time. Okay, what was the purpose? My prompt after this architecture, I created several instances. I tested multiple instances. These are illustrative numbers and we'll create the baseline when we start our project. But you have to map it out if it is truly helping you or not.

Not as a critique. Yeah, it can act as a critique, but it does not tell a smaller model to fix it. See, critique is what? I can critique you and then you'll go and fix it. But it does not as a critique. It will check it. If it is not correct, it will replace it, which means I will not give you advice. I will check whether you have corrected. If you have done wrong, then I will take that. I'll only fix it. I'll not ask you to fix it. Otherwise, it increases time. Again, the request goes in, prefill has to start, decode has to start, those kind of things. Critic is not a right word.

Okay, read this very nicely. Working with Claude, which calls produce long and predictable text? Usually you want it to— you want— you put your smaller models where we value speculative, where model calls are more predictable, and whether a draft model worths its extra memory, because it will take this extra memory to store the KVs and KVs, basically working memory, it will take it. What is acceptance rate? That how do we check when we want it to do the prefill on its own? If it is not helping on your test cases, you will remove it. If speculative decoding on the evaluation set is not helping you, and what is evaluation set?

Evaluation set is handwritten set of ways your users will use it. The golden dataset. And you do not want your model to be failing on your golden dataset and also overfit. You have just made for it to work on your test set. Then how So that's why a lot of times we do not even look— whenever we develop

agents, we do not create evaluation sets. A separate team creates it and we do not even know. Otherwise, our developmental things will be around the test set. Whatever improvements we wanted to improve on our validation set and generally do better on test.



vLLM: four speculative tokens using n-gram proposals

```
vllm serve Qwen/Qwen2.5-7B-Instruct \
    --speculative-config '{"method": "ngram",
        "num_speculative_tokens": 4, "prompt_lookup_max": 4}'
```

vLLM: inspect speculative-decoding metrics

```
curl -s localhost:8000/metrics | grep spec_decode
```

What is being stored in GPU memory?

You have fine-tuned your own model. Then, and then you want to serve it. But whenever using an API, all of this is not required for you. You just end up paying the per million tokens kind of thing. But when you— we have almost completed inference architecture, but what happens in this block, where it runs on and what is being stored and how it is being stored, and how we can fasten up our prefill and prefix, and what are the strategies which current companies are using to fasten up their prefill and decoding phase. What

happens is, just to read this thing, you have 50 incidents. And they arrive in like, let's say, 10 minutes.

And incidents is what? Incidents is something broke. Every investigation can grow up to 20,000 tokens, 20K tokens, and your GPU has around 48 GB of memory, and your vLLM utilization thing is up to 90% of the GPU memory which you can use. The inputs can be about 21 investigations because that much amount of thing, because if you consider if one token can grow about 20,000. Based on the thing we said, okay, vLLM will expect 21 investigations because the utilization we have set to 0.9. Then now, once the investigation keeps on going, you have another keep on coming into the queue. In that, we have at least these are the— there are more actually, but at least broadly, if you look at that, it eventually contains the following architecture.

But there's a question is, why can 48 GB of GPU memory only hold 21 investigations at a time where it is busy? And is it any way that we can increase from 21 investigations to probably 30 investigations or increase it further? That's the goal. Your— if you use API tokens, it is not going to be much in your calculation. It is not going to be much in your, you won't mean— you may not work in this part. But when you serve your own fine-tuned LLMs, then suddenly, let's say that you have built your sales language model, it— you have X amount of GB of memory, you need to be able to say, okay, this is how— because you'll, based on the calculations, for example, if you have X number of weights.

Every weight is stored in BF16 format, so 2 bytes. That is going to be X amount of memory. Now, okay, you say that 20,000 tokens can be for per investigation. I'm going to have maybe 20 investigations at a time. That is going to be your KV cache pool. Working memory is where you're calculating those attention scores of FFNs and things like that. And kept free is 5 GBs in this case. Basically waits in the, at least in the entire state. You have what, why we are doing this. First of all, to understand it concretely, we are doing this for one smart reason is right now it says our AI SRE is busy.

Can we work something to increase from 21 to 40? Or at least 30. How can we increase from, so what is the logical step? If we want to increase the number of investigations from 21 to 30, yeah, priority queuing and batching, that is one we'll we'll get to. But is there any other?

You have weights which are fixed up here in one of the models. It does not change. The KV caching pool is all the running investigations up to 20,000 tokens per investigation. That is the pool. And this can be of all the running investigations. This is where your 3 GB of memory will be stored for the working memory and kept free is what we want to use. And as I said, vLLM uses 90% by default and little is left for the safety or anything which you go overflow. For example, a particular investigation grows more than 20,000. Okay, so what goes into the working space like 3 GB memory which we stated?

Attentions, the scores, the feedforward network, whatever we do the calculations. Right now, which means whatever the calculation which you're doing at the moment, it goes inside that working memory. You are saying that, okay, you want to serve the GPU memory utilization 0.90%, and by default it uses the 90%, and you can also control how much utilization you wanted to give. But here is the important part. Right now we are saying that, okay, you are allocating 16 GB For weights, XGB for KVs. But how are my weights being stored? How are my— if I have 1.75 in weights, how are these stored?

Is it stored in a literal 1.75, or is it stored in some format into your computers which you call as binary numbers? Right now here, your each weight—for example, let's say that you have over 16 numbers, and this is the example which we'll use. Let's say you have, following 16 weights up here. And let's say that every number can be represented by 2 bytes each, which means 16 bits. Total storage it will take for our 16 numbers is 32 bytes. Imagine you have around 7 billion parameters. 7 billion can be multiplied by 2 bytes. That is how much storage it will need to convert to GBs further. That is the storage it will need to store that particular base, our model numbers.

Why only 21 investigations fit

Memory item	Calculation	Result
GPU memory	48 GB x 0.90	43.2 GB usable
Weights	7 billion x 2 bytes	14 GB
Working space	Illustrative allocation	3 GB
KV pool	43.2 - 14 - 3	26.2 GB
One investigation	20,000 x 60 KB	1.2 GB
Full-length requests	floor(26.2 / 1.2)	21 running; 29 waiting

Rounded illustrative decimal units. Actual startup allocations depend on the model and runtime.

Bits, bytes, range, and precision

The goal here is to have more running investigations at one point of time. If we are storing one weight in 2 bytes or 16 bits, can we store it in 8 bits? Can we store it in 4 bits? Let's say 0.12, I store in 16 bits, which means 2 bytes. What if I store 0.12 in 8 bits and then convert it back? Less precise, because we are giving it less storage. Previously we were storing it in 2 bytes. But now we are storing it in 1 byte. If you are going to store in 1 byte, then 16 multiplied by 1 is just 16. We have done a 50% storage reduction.

The trade-off is loss of information/precision. It may give me 0.10 after I convert it back. Here the error is 0.12 minus 0.112 is a little less. The trade-off is that you can store a particular number in 16 bits or 8 bits. We can achieve the goal of storing our weights in fewer bits by using a format with fewer bits. But when we do that, we will lose precision.

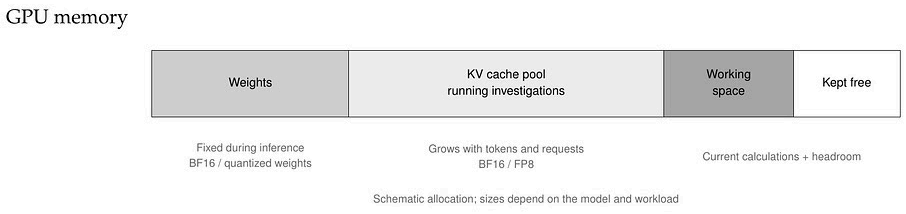
Here the range is much higher. Here the range is extraordinarily high. It is able to store this as well, but the detail decreases. You see our original number, when we converted it back, it is 256. The detail has decreased, but it's fine. The error is only 1. Here the error is about plus 0.05 or something. But here it is able to represent the bigger numbers as well. If your weight was 70,000 at some place, FP16 cannot store, but BF16 in that format is able to store. But then you have our nicest FP8, 1 byte per weight. Here you lose significant detail. Here it is able to store 257 in this manner, and it is too big to store 70,000 more.

Appear. But here we have a significant storage loss. The reason why I'm telling this is when you convert your number back to your— when you use

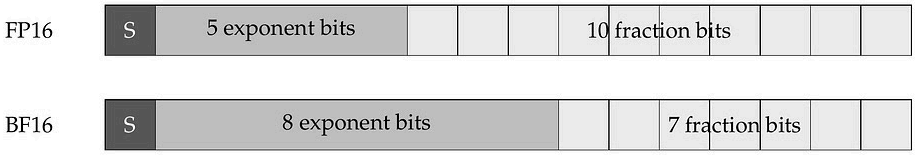
different format, we can solve our entire problem of reducing our memory by choosing the right format. But they all have a different trade-off. I have questions now. Range and precision. If I have to store my weights, these weights, what happens if I store FP32? I will increase my precision and range. I'll be able to precisely store whatever. When I convert it back, I'll get what I stored, what my model learned. But what will happen because of that? 4 bytes per weight.

What is FP8? It is taking 4x less memory than FP32, but we are losing significant detail or precision. Okay, let me just repeat this once again by questions. If I want to store 0.73 and I use FP32, more precise when we convert it, when we store it? How is this being stored in this binary number. Will when I store using FP32, will it be more precise or less? More. What is my trade-off? More memory or less memory? More memory. Now I want to store 0.73. Will it be more precise or less precise? As compared to the previous one using FP16 format, less precise as compared to the— but will it be more memory or less memory as compared to the previous one?

It is storing 2 bytes per— Less memory. Now I want to store 70,000. I use FP16. Will I be able to store? Because the range is up to 65,000 number only, so it cannot store. Range is not supported. But BF16, can it support or not? Lesser memory than FP, but you can support such. Now I use 0.73 using FP8. Extremely less precise. You see, it became 0.75. Range is just 4481.



The same 16 bits can be used differently



FP16: more detail, smaller range.
BF16: wider range, less detail. Both use two bytes per number.

S = sign. This changes the allocation of bits, not the total storage.

Store a number, then read it back

Format	Bytes	0.73	257	70,000
FP32	4	0.730000019	257	70,000
FP16	2	0.729980469	257	infinity
BF16	2	0.73046875	256	70,144

Storage conversion example from the book. Range and precision are separate choices.

PyTorch: inspect bytes per stored number

```
import torch
W = torch.tensor([[ 0.12, -0.35,  0.08,  0.21],
                  [ 0.42, -0.16,  0.31,  0.05],
                  [-0.09,  0.27,  0.14, -0.22],
                  [ 0.18,  0.06, -0.41,  9.60]], dtype=torch.bfloat16)
print(W.element_size())              # 2 bytes for each number
print(W.numel() * W.element_size())  # 32 bytes for all 16
```

PyTorch: compare the stored formats

```
import torch
x = torch.tensor([0.73, 257.0, 70000.0])      # FP32 by default
for fmt in [torch.float32, torch.float16, torch.bfloat16]:
    y = x.to(fmt)
    print(fmt, y.element_size(), y.tolist())
```

Seven billion weights: raw data storage

Precision	Bits per weight	Raw weight data
FP32	32	28 GB
BF16 / FP16	16	14 GB
INT8 / FP8	8	7 GB
Four-bit codes	4	3.5 GB + scales / metadata

Decimal GB; ignores non-weight runtime memory. Quantization formats have additional metadata.

Quantization: a ruler with fewer marks

What is our goal? Less memory. Less memory utilization per token. Now, so that less memory, how it will happen? We are storing 16 bytes per array. 2 bytes equals 16 bits. 1 byte equals 8 bits. We are storing using BF16 format. That is taking 2 bytes. To reduce our per weight, we can use— we can just make— if we can make this to 4 bits, not byte, 4-bit, which means even half of a byte, we are going to reduce our memory bit over here. 4 bits, not byte. If right now it is using 16, I can make it to 4 bits. I am storing as 16. I want to make it 4 bits.

What will happen? Can I make it? Yes, I can make it. But what will happen if I just directly make it? Forget about less precision. I'm going to use all my numbers. Forget about less precision. The model will not make sense. Entire thing, it will not make sense. The goal is very clear to us. Less memory utilization. Right now, one weight is stored as 2 bytes using BF16 format. 2 bytes is basically 16 bits. If I can make 16 bits to 4 bits, Will it make sense for us to go ahead and— that will significantly lower our memory utilization, then thus increasing the number of investigations if we use our own fine-tuned models.

What can be the strategy? 4 bits can contain how many possible patterns? Okay, 2 to the power 4.

It will give us 16 patterns because it goes as a combinatorial thing. My 4-bit ruler will go from -7 to +7. If you have 16 bits, there can be 65,536 patterns. There can be multiple patterns which you can draw.

What I said is I will, because I want to, that is a technique of quantization. What I'm doing is I'll have a ruler because I'm using 4 bits. I can have a ruler with multiple marks also. If I'm using 16-bit, then that can have 6. The marks will be around 6, total of 65,537 marks up here. INT8 will have around 256 marks into a thing. Similarly, 4-bit is going to have around 16 marks. Over here, from -7 to +7, I'm going to find the biggest weight. We decided on— here is our weight example, running example. Take a screenshot if you can. Take a screenshot. This is our weight. From here, if I want to quantize this row, I'll take the biggest weight up there, and I will make sure I'll mark the ruler -7, which is basically -0.35.

Find the biggest weight, and then I'm going to make sure that the 0.35 is at the highest item, and then we have to put the sign also. Then you choose the scale. Basically, the scale is being chosen how? You basically, you say, okay, the scale would be whatever is your highest number into that row, which is -0.35 divided by the 7, because over here we have the -7 to +7. It is not normalized right now. We are using dynamically number over here. Then whatever nearest comes, for example -35 will be automatically written as -7. If you convert that weight, for example, if our weight was this— where is our weights?

If my weights are following from our first row, I'll first of all— this scale is one of the not hyperparameter but calculated parameter, nearest mark. If you look at, if you divide by the total which means your quantize, ultimately it is coming around 2.4. 2.4, the nearest mark over here is 2 for 0.12. You will end up writing 2. Now if you make it come back, you multiply with the scale again. When you multiply with the scale again, you are again going to come back 0.10. Here, how much precision did you lost from the original weight? When you take that weight, divide by the scale. How did we reach to the scale?

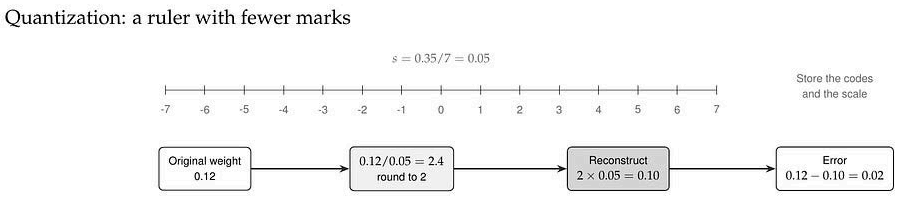
We taken up our score and then said that 7 divided by over high— your highest point. Sorry, 0.35 divided by 7. Ultimately came to 0.05. That is the scale do we have between each. When you take that weight, divide by your scale, you're going to get the quantized. Then you, based on the ruler, you have 2 over here. Now, which is nearest? This particular 2.4 is nearest to 2. Your stored weight, the new weight which comes after quantization, is 2. But then when you want to reuse it back, you want to— this time without dividing,

you want to multiply with 0.05. Your previous weight was this. After quantization, it came to 2.

Now this can be stored into your 4 bits. Because 4 bits can take range from -7 to +7. -7 to +7. This too can be stored in 4 bits. How do you convert it back? When you take that mark and then multiply with 0.05, that will come around, 0.10. That is the lost precision, is 0.02.

We take the largest 0.35 divided by 7, we get our scale 0.05. Then you divide your weight 0.12 by 0.05. You store the scale also. To store the scale, we are using a 16-bit BF16 format. We cannot lose the precision of the scale. These numbers are whole numbers, I can store them in a 4-bit format, but the scale also has to be stored. To store the scale, we need to store it in high precision. Total occupation is 32 bits for that single row.

Okay, let's understand this with one more example. Say if you want to understand— sorry, if you have 4-bit patterns at 16, you can use -7 to +7. This gap between— for the first row was 0.05. If you use scale, and then for a 7 billion parameter model, that is going to be 4 GB occupation. For INT8, more precision, you're going to store more. When I say more precision is what? More precision is less error in all the thing. Over here it will store 7 GB. If you use 6 GB, we have 16 total weights are 14 GB. Just by quantization, you have reduced from 14 GB to 4 GB, thus incorporating multiple investigations.



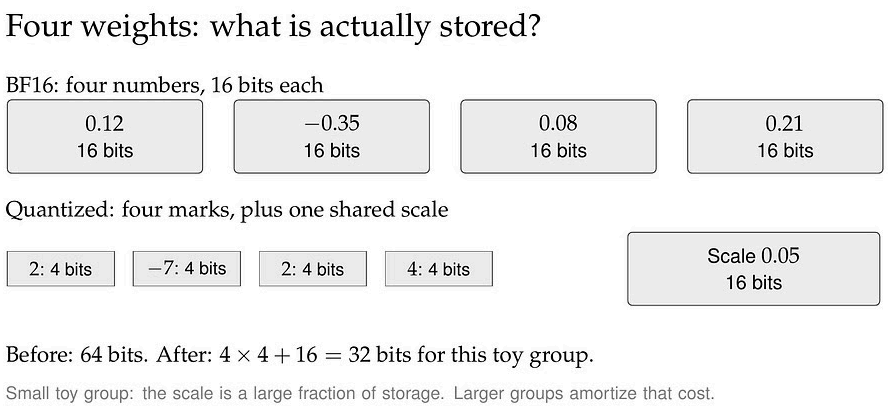
Quantize the whole row: scale = 0.05

Weight	Divide by scale	Stored mark	Reconstruct	Abs. error
0.12	2.4	2	0.10	0.02
-0.35	-7.0	-7	-0.35	0.00
0.08	1.6	2	0.10	0.02
0.21	4.2	4	0.20	0.01

Illustrative weights. Store the marks and the shared scale.

PyTorch: quantize and reconstruct the row

```
import torch
row = torch.tensor([0.12, -0.35, 0.08, 0.21])
scale = row.abs().max() / 7     # 0.05
marks = torch.round(row / scale).clamp(-7, 7)   # 2, -7, 2, 4
back = marks * scale            # 0.10, -0.35, 0.10, 0.20
```



Small toy group: the scale is a large fraction of storage. Larger groups amortize that cost.

What goes wrong with one scale?

How is scale being decided? 0.05. Scale is being decided by taking the largest number in account in that particular row. That's 0.35. Large variation in weights will spoil every single thing. Let's— what goes wrong? Let's say that you have weights stored in BF16 formats like this, 0.12, -0.35. Now you need to— you want to quantize everything. What you will do? You will take the largest weight. Do the calculation, whatever the calculation we did. We will take the largest, find the biggest weight, choose the scale. Basically, we'll

take the largest weight, which is 0.9160, and then you want to divide by the 7. Then you will ultimately get the final one.

Then you find the each weights and then put the matrix. What ultimately we'll do Is we will do that, but the nearest one, everything will become, which means 0, 0, 0, 0, 0. All the weights, if we store using 4 bits, they have lost the precision, and which is going to be stored like this. When we convert it back also, I will be only be getting back my precision, this 9.60, 9.60. Everything else will become 0. Because why? We have chosen the larger weight in the entire row. Larger weights can destroy my variation because that's the scale I'm looking at whenever I'm looking to take— generate the scale value. 1.37, 1.37.

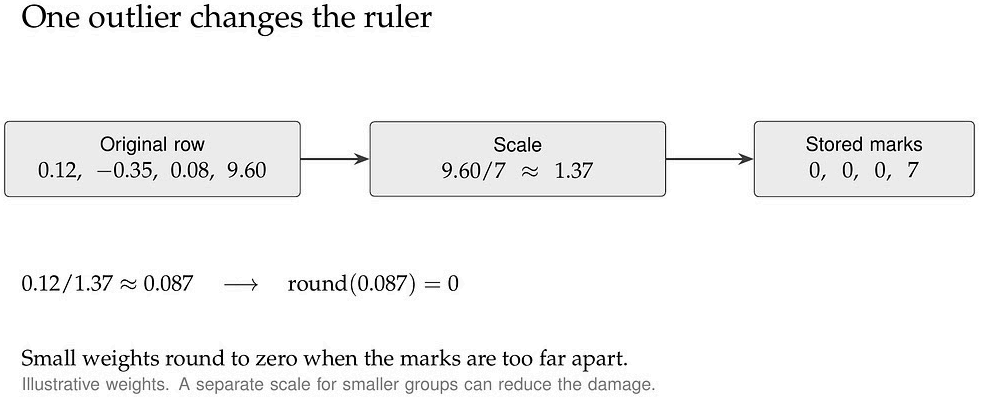
This becomes the overall problem in quantization. Why am I saying even you quantize, when you quantize, you ultimately going to have 9.16 Up here, because that's the largest number which you're gonna take. You're gonna take the largest weight, and then you want to do follow the process to convert that to a 4-bit codes. But when you multiply it back, you have lost all the precision. The only precision you keep is this one weight. That will obviously will not help us. How do we do that? How do we fix the problem?

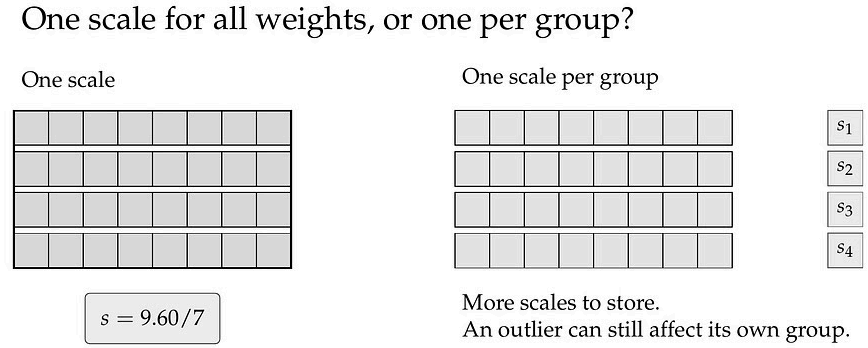
But think about this. Yeah, there are ways. There is AWQ, GPTQ, and for a lot of techniques. In our previous example, we did not take in the largest weight of all. We just grouped it together. We said I will do the quantization per row. Here we just take in 0.35 as the largest weight. Here we will take 0.31 as the largest weight. Here we'll take and then quantize per row. For this row, I'll have scale number 1. For this row, I'll have scale number 2. For this row, I'll have scale number 3. For this row, I'll say scale number 4. Scale is being stored at 16 bits. Can I do that grouping of rows-wise, rows-wise quantization?

Yeah, we can do, we can do that. But what is the trade-off? Multiple S, multiple S, and also losing precision as well. What if in the row itself we have different, a larger weight? Column-wise we can do. Good idea. You can do column-wise, but what if some weights are outliers? You don't know. You can have those outliers as well, row-wise, column-wise. A big strain on GPU also to do the multiplication also. Exactly, there are multiple problems with the group-wise, and, and also more importantly, you will store S's. You'll keep on

storing S's, not standardized. When you convert your thing to normalized, then you have to convert it back. You will lose information there also.

Lot of strain on the GPUs also. Normalization is a good technique also in the quantization. First fix is you group it. What you do? You simply do the row-wise. Here stored big codes, then you convert it, you lose the less precision. But here, if you see, last row again got the same problem because one of the rows was an outlier, 9.16. If you do with group-wise normalization, technically you can do to make sure why the thing is not working.





Activation-aware weight quantization

But yeah, this fix is pretty easy also. One way was we said I will quantize only one row, but this row also faces the same problem, outliers. What I said is let's look at one example. What is output dependent upon? A weight importance depends on the input it is multiplied with. Think about it. Your house price exam, house price prediction also, you're understanding the

effect of how much square feet matters in the final prediction. When the, when the weight gets multiplied, if the input is higher, when it gets multiplied, you're going to finally get a possible answer right up there. Can we say that a weight importance depends on the input it is multiplied with?

Over here, if we have input 1 multiplied by weight 1, your final strength is finally output is significantly determined by inputs, input 1 and input 2. Your final cost. Can we do that? Let's say our weight 1 over here is after quantization and things like that, we reached to be 0.14 over here. And then when you multiply over here, which means your total 10, with 0.14 plus finally your 1.80, your final output which comes, which finally comes is 2.20. But when you reverse it back, that is around 2.80. You are wrong by a total of 0.60. Let's assume that you have a ruler. I'm going to say that seven and four.

Your original weight is zero point fourteen. When you quantize, that becomes zero point maybe twenty. When you convert data, what do you? This is not going to change. 10 multiplied by 0.14, that is giving you 2.20 answer, your input. This seems small, your weight seems small, so your— when you quantized, this became 0.20, but the strength is— the input is 10. When you multiply 10 minus 0.20, or the same thing, let's say, over there also, it becomes 2.80. From 2.20 to 2.80, that is a difference of 0.60. How did it happen? Even though my weight was small, it happened because of that input. If my input would have been lower, then I could have done a good job.

My error would have been less. If my input was not 10, it was, let's say, 2.5, and my weight would have been higher, multiplied by something, let's say 0.5. I'll come to that. How do we do that? I'm just taking an example. Then my total output would have been, let's say in this case 2.30. We would have been wrong by only 0.10. Idea is, if my output is dependent upon my input, if my total output is dependent upon my input 1, can I somehow decrease my input and increase my weight? Activation-aware quantization, that is one of the techniques. And how will I decrease my input? We are using 4 bits.

I will divide my input by 4.

And I will multiply my weight by 4. That's what it is doing, is basically saying that activation-aware is saying that It makes the important weight bigger before rounding because the input is bigger and makes the input smaller by

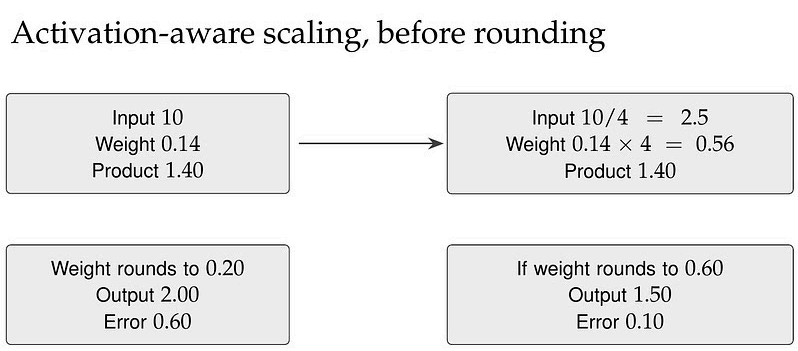
the same amount. Here we are talking about 4 bits, so the output is unchanged before rounding. Now when you do the calculation, you ultimately get 0.230. Why are we doing this logically? Also, if you think about it, think about it logically. Imagine your input is higher. What does it mean? It means that it, that it, that your weight importance is determined by your input. We understood that input is important. What we did, because we understood, I'm not going to do that for this guy because the input is lesser there, the impact is lesser.

But the weight importance is determined by input. By that, what I'm going to do, I'm going to reduce the input by some factor, and by the same factor I'm going to multiply the weight to be able to quantize. The error is much smaller. Activation-aware weight quantization, which is the very famous one right now.

You don't start to quantize first. You look at the entire matrix first. You see which inputs are outliers as compared to other ones. You scale the input by a certain factor and increase its weight by a certain factor. Then you look at another row, there is another bigger input. But you have to look at the entire matrix first, known as calibration. First of all, calibrate which inputs I should select to scale up and down. First of all, you run through real inputs. You want to run your AI SRE through real example prompts because every input prompt may be different, and then understand which inputs I should select to scale down by a certain factor and multiply their weights.

For AI SRE, examples should be real investigations so that we can run and identify which inputs matter, and then only we quantize them so that we can retain their precision. GPTQ is another such technique. And then you have NF4. AWQ is widely used, and NF4 is also widely used.

But if you're using your own fine-tune example, is you can use things like 100 to 200 such real examples. 100 to 200 depending upon how much bandwidth you have. But essentially, vLLM can serve using AWQ or another quantization technique.



Illustrative rounding: factor 4 is a scale choice, not a rule implied by four-bit storage.

vLLM: serve AWQ weights

```
vllm serve Qwen/Qwen2.5-7B-Instruct-AWQ --quantization awq \
  --gpu-memory-utilization 0.90 --max-model-len 20000
```

Weights, KV cache, and working precision

Similarly, you have KV cache pool. We have a big word known as weights over here. Then we have KV cache. Then we have working memory. Right, we say we were able to convert BF16 to 4 bits using quantization. Can we do something with KV cache also? Right now it is, it is also stored in BF16. Can we also change it? Can we change it? Same method. But here, do we want more precision or less precision? We want more precision. I will store it using 8 bits using instead of 4 bits. Working memory I'll store is using BF16. As soon as I convert from BF16, I'm going to 50% reduction. Right, see, the reason why I'm telling it's completely your decision based on your bandwidth requirement.

See, tomorrow you are given only 48 GB of memory, you might want to explore different ways. See, a lot of this is a format which is widely used, is people will tell, hey, use 4-bit for weights. Use eight bits for KB and use BF16 for working. You should understand, okay, why are we doing four bits? Is because I can locate to less precision and more. I do not matter weights, but my current working KV caching should be having a little more precision, and a working memory definitely should be having more precision. BF16. That is why we have different formats for different weights being stored.

But KV caching, we can reduce it. KV caching, we said 4-bit, well-scaled FP8 notebook. It was also storing 4 bits at once. The number of investigations increased from 30 to 60 investigations just by having more investigations to come in by lowering our work KV cache memory. But 19 of them we were able to correct out. We were able to Get our correct answer. If you were not able to do our correct weights, which means more we were able to compromise on the storage of bits of our KV cache, same investigation, but we lost our baseline.

But over here, what happens at every token? The problem was that from BF16, we reduced our weights 4 times lesser memory, but our speed was only 2x faster. We reduced the number of tokens which was stored, but our speed only increased 2x faster.

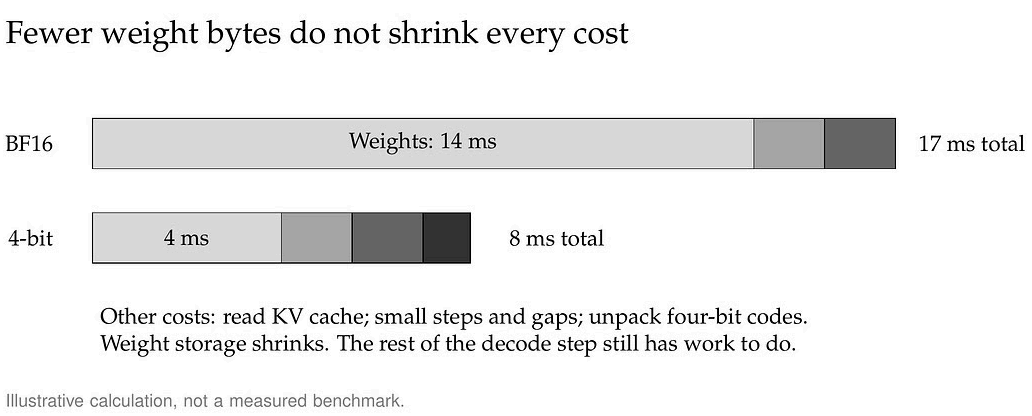
Memory savings still need a quality check

Weights	KV cache	Correct examples
BF16	BF16	20 / 20
Four-bit AWQ	BF16	20 / 20
Four-bit AWQ	Well-scaled FP8	19 / 20
Four-bit AWQ	Badly scaled FP8	6 / 20

Illustrative comparison; establish a baseline on the investigation workload.

vLLM: choose FP8 for the KV cache

```
vllm serve Qwen/Qwen2.5-7B-Instruct-AWQ --quantization awq \
  --kv-cache-dtype fp8 \
  --gpu-memory-utilization 0.90 --max-model-len 20000
```



Keep the tool results short

Your call number 1 was 4,000 tokens. Why does it increase every call? 7,000 tokens, 10,000 tokens, 13,000 tokens, 16,000 tokens. Why does it increase? One investigation. Context builder, correct. Usually in our architecture, your harness should decide what should go in each input. For example, you can keep the stable start. For example, you can just have prefix caching enabled so that every call uses the caching. Then you have to keep the tool results short. For example, you want to be able to Loggings, whenever it has, you summarize using some open models. You summarize it using some powerful open models, or you can do maybe only show the relevant lines, or then you set a budget.

For example, if a particular token call exceeds X amount of tokens, you want to remove or compact the past tokens. Like right now, no, I'm in compaction and then Claude. Has a compact and things like that. They work on a similar mode. You can set a budget. For example, if any prompt passes, then oldest tool results can be shown with one-line memory rather than completely removing. But the trade-off is that you may lose a lot of work. Similarly, see, ultimately you want to do the multiple such harness improvement up there. And here what we are doing is we are ultimately doing multiple summarizations so that context issue is solved.

PagedAttention: give memory as it is needed

We said that a particular investigation Is twenty thousand. One investigation is twenty thousand tokens. Second investigation reserve. When I say twenty thousand tokens reserve, in our KV caching, that's how we decided. No KV

caching pool is let's say twenty GB. That can support around thirty such investigations. Each investigation up to twenty thousand tokens. One investigation came and just used up to six K token. One investigation came and used up to 3K tokens. Third investigation came and used 19,000 tokens. We have left rooms. No, is that going waste? That is going wastage. We reserve 20,000 but only uses is 66K. Memory which was reserved, it is not being fully utilized. For example, let's say several people who you check in hotel around 5 PM and then you check out at 5 AM in the morning.

Check-in policy of new people is around 2 PM, but you can reach out to the new customer and say that, hey, okay, you can also check out 10 AM by additional charge. Similar analogy. That bracket is being faked. Nobody knows what to do with it right now. What's happening is because your every tokens were around 20,000 per tokens, but averaging we are having around 60,000. 70% of the reserved memory holds nothing. That is a big of an issue, which means memory that is being reserved but never used is known as fragmentation in general.

What can be done? Let's say I have a huge memory. We reserve 20K, 20K, 20K, etc., etc. In that time, 3 investigations can go on. Can we do something here? Think naturally. What can be done if let's say 6K is on an average? Can I do one thing? Instead of 20-20K reservation, I can create pages. Each page will do 2K tokens. Anytime a new guy comes, I can just give them 2 pages. If it requires 4 pages, I can give 4 pages. Each page gives me 2K, 2K tokens, 2,000, 2,000. If it comes as we require, I can just give them 1 page. 1 page is about 2K tokens. I can give them another one which will be 2K tokens.

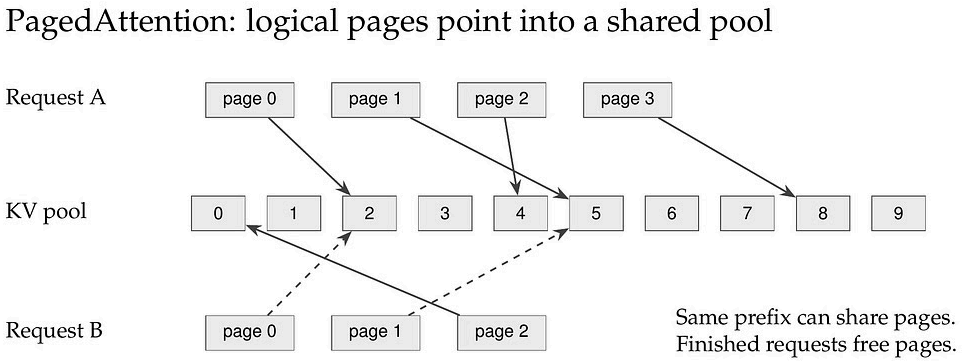
I can give them another one, 2K tokens, all the way down to the end. By that, what I'm saying is I do not need to give them reserve. I can convert my memory into blocks, into pages. Each page will represent some memory. Anytime a request comes in, they can take the request, take maybe 2 pages initially. If it requires more, they can take more pages.

Known as PagedAttention. The KV cache pool can be cut into small pages of 16 tokens each. Small. Which means 16 tokens is how much? 16 multiplied by— if we— if our per weight is per token is 2 bytes, Then 32 bytes per page, small block. Now each investigation gets a page when it actually needs one.

One new page carries 16 tokens. We can give those pages to them and then they can keep on using this PagedAttention. And then we have ultimately you want to use block size, as I said, is 16 over here. You make that happen. Quantization technique 2, AWQ, GPU memory utilization up to 90%. And over here, what you're saying that you just give it 16 tokens.

Instead of reserving, you just go ahead and give it in blocks so the new investigations can keep on taking such requests.

It is in general, flash attention is rather than coming, storing your weights into HRAM, you again coming here, you again doing some calculation, going back and taking stuff from there. You take the block there itself and then you do the calculation and then you give it back. Doing a lot of back and forth, you just take the entire block and do the things here. I'm not going to cover that right now, but at least the page attention, what it is doing, Flash attention is built in vLLM infrastructure. It is by default, and you can explicitly remove it. But KV caching, to manage that reserved thing, I'm saying give it in blocks, 16 tokens each block, so new requests can keep on taking the requests pages.



Illustrative block map. Each block in the running example holds 16 tokens.

Static, dynamic, and continuous batching

Static, dynamic, continuous batching. What is that? In static, you wait for the full bus and everyone leaves together. For example, one investigation, second investigation, third investigation, fourth investigation. All of that can wait for a bus and then leave together. When I say in everyone, when they complete each other, then they only leave. Dynamic is leave when full or after a short wait. Means I will wait until everybody has completed. But here, let's say this has completed, this can leave. But here, these 2 have not completed,

but the slots are still left of this. Let's say if there is total 3 slots, and this will be understood, here is dynamic says you have 3 investigations at a time in a GPU.

I will start a little later, or I will leave a little later, but still it has to leave only when everybody is completed. Or if this has completed, this has to wait for this to complete. What it says, I will start once everybody has sat in the bus, time limit, or after a short wait, or leave Only after a short wait, but still we are going to hold this position. But continuous means continuous generally, which one of you stated, I can just go ahead and once this is completed, somebody else can come and sit. It's like a running bus. People can come, sit, go, etc., etc. Continuous batching is basically a finished investigation leaves at once and a waiting one takes its place in the next step.

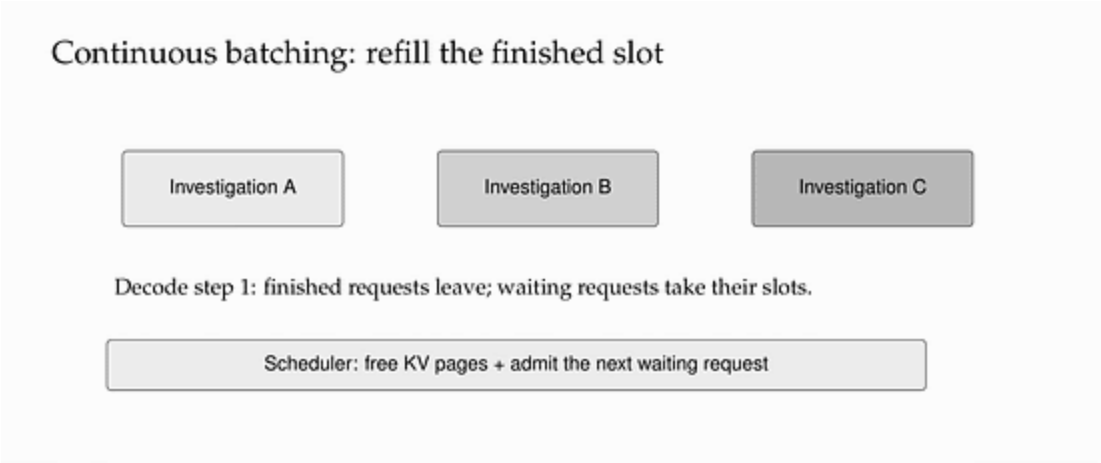
If this has been completed, somebody else can come and then complete that thing because your every investigation, some investigations can take more time and some investigations can take lesser time as well.

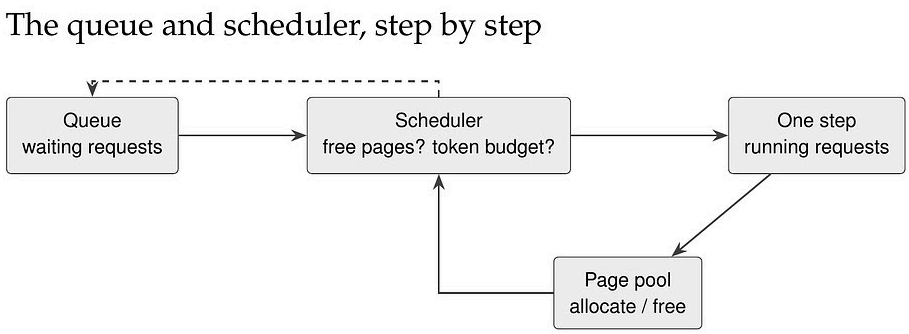
What you are seeing over here, you want it to either sit in the bus and then wait for everybody to complete and then leave. But in that time, some investigations take longer, some investigations take less time. Continuous says after it has completed, it can leave and another one can come and sit in my place. Continuous batching, which your vLLM does it precisely, which is your queue and scheduler mechanisms. And queue schedulers, I think someone has asked, is look at the queue of waiting requests, admit new ones while there are few pages and step tokens is not full, which means take as much as you can. Until you have completed your 16, 16 tokens and then queue those people who have not yet completed.

You run one step for everybody admitted. If somebody has completed, you can queue the new one, then free the pages of anyone who has finished so that a new one can eventually come.

If you want to serve the model, let's say some model, And if you're using your own fine-tuned model, you want to use some quantization technique. Why are we using quantization? Why free up space? To have more investigation. Why are we changing the KV cache to FP8? To get more investigations. Enable prefix caching. We can use the caching technique. Max token model

length 2,000. GPU memory utilization 0.90. Number of sequences that can go in inside it is basically 6,664 tokens in general, 6,664 sequences in general. Max number of batch tokens is 8,192.





More throughput can mean a slower single answer

Investigations	Total tokens/s	Tokens/s each	TTFT
1	125	125	0.4 s
8	800	100	0.6 s
32	2,200	70	1.1 s
64	3,000	47	2.5 s

Illustrative throughput trade-off; actual values depend on workload and hardware.

vLLM: the combined inference-service command

```
vllm serve Qwen/Qwen2.5-7B-Instruct-AWQ \
  --quantization awq --kv-cache-dtype fp8 \
```

```
  --enable-prefix-caching \
  --max-model-len 20000 --gpu-memory-utilization 0.90 \
  --max-num-seqs 64 --max-num-batched-tokens 8192 \
  --speculative-config '{"method": "ngram", "num_speculative_tokens":
4, "prompt_lookup_max": 4}'
```

The combined vLLM command: each setting

Setting	Running example	What it controls
Quantization	awq	Weight format
KV cache dtype	fp8	Cache format
Prefix cache	enabled	Reuse shared starts
Model length	20,000	Context budget per sequence
GPU utilization	0.90	Runtime memory fraction
Maximum sequences	64	Concurrent sequence limit
Batched-token budget	8,192	Token work admitted per step

Example configuration from the book; benchmark and validate on the target hardware.

What should we measure?

And finally, you want to test following things: time to the first token so that you can debug your prefill phases, time between tokens, how fast your answer comes, throughput, Is how much total work does the GPU do tokens per second? How many investigations can run at the same time? KV cache usage and queue length. How close are we full? Is anyone waiting? How many correct diagnoses per minute? Anytime you introduce your new architecture or anything of that, then you may want to measure at least around these. Tokens.

And then your GPU memory, we have converted your weights to 4-bit using quantization technique, KV caching, again FP8 using FP8. And then you want to measure your metrics, which are time to first token, time between tokens, how much KV caching will be used. These are the kind of metrics which you ultimately want to use, at least from the inference service for our project till now.

vLLM: benchmark a long-input workload

```
vllm bench serve --model Qwen/Qwen2.5-7B-Instruct-AWQ \
  --dataset-name random --random-input-len 8000 --random-output-len 200
\
  --num-prompts 200 --max-concurrency 32
```

vLLM: monitor running and waiting requests

```
curl -s localhost:8000/metrics | grep -E "time_to_first_token|num_
requests_(running|waiting)|cache_usage|preempt"
```

A smaller specialist SRE

Do you think that it has a scope of Fine-tune model, our AI SRE. What if an organization wants a fine-tuned model? Can we have some LoRA adapters on top of it? We can have it. Basically, it is saying that, okay, you can actually do in 3 steps. That is a ladder. Better prompt, give it facts, or fine-tune the model. That's what it says, a smaller specialist SRE. And we do not in this project create a smaller specialist SRE. We focus on agentic engineering. But if you can arrange data and be able to fine-tune in the same manner, whatever we have done, nothing changes there. Is if you want to teach the format, the style it should take, and the right tool choice, you may want to fine-tune.

If the job is very narrow and you can potentially come and train a model where it inhibits a particular behavior, or if the requirement comes along, then you may have to choose the fine-tuned models over here if the model behavior is up there. If your architecture stands up with an SRE, a specialist model, then instead of an API, you may end up using a LoRA adapter. In this project, we don't do it. It's more about data.

The question should always be, first of all, identifying if an organization have their ways of doing things, if they have enough data. Otherwise, LoRA adapters or fine-tuning would not make sense at that time. The first and foremost question of training an SRE is basically coming through a point and saying that, okay, do you have data when your human SRE used to do

something? If that's the case, I can identify one repeated behavior and do something for you.

We say that we take some examples based on the personalized before every call. We say contract review, claims intake, service operations. Then what is the current bottleneck? Is your human review? If somebody is doing a repeated deal, he repeats checks same kind of decisions. More cases, more review time. SLMs fits a repeat task with enough volume and good examples, which means high-volume things are there. There is a defined task and we have reviewed examples.

A trained model inside a working business process, is review as-built data pipeline, test on unseen customer release approval, task train SLMs, application checks, allowed action review. We also focus on agentic engineering part. What is the handover, a model, a test, integration code, and how to run it? SBL brings its own frameworks and tools to your business processes. We have created Palantir ontology reports to be able to understand a particular customer, understand the overall data which they have, take out the best 500 to 1,000 such examples for us to go ahead. And then we back it up with Palantir's reports and then keep the knowledge your team puts. Basically we talk about agentic engineering also.

RAG, Guardrails, PII fixings, all of that we talk about. We just don't talk about SLM. SLM is one of the core of it. SBL takes the responsibility for the build from training data to application handover. You can buy a model API, commission fine-tuning, build internally. Where the responsibility comes in, engage with SBL. I will take care of your data. I will take care of your model. I'll take care of the integration, and I will take care of the handover. And then we talk about the first POC. If something goes wrong, I will revise or improve. What we measure: task quality, review time, response time, and total cost. We focus heavily on the tasks.

How will you close the loop? Agentic engineering. You give me just a chatbot that can answer questions, that can answer the next, sentence, but you need to put it in production. Guardrails, how do you identify? There are multiple agentic rules over there. Which we have not gone through, but there is a harness around it.

vLLM: serve a trained LoRA specialist

```
vllm serve Qwen/Qwen2.5-1.5B-Instruct --port 8001 \
  --enable-lora --lora-modules sre-tools=./sre-tools-lora \
  --max-lora-rank 16
```

Ayush Singh is free today. But if you enjoyed this post, you can tell Ayush Singh that their writing is valuable by pledging a future subscription. You won't be charged unless they enable payments.

Pledge your support

LIKE COMMENT RESTACK

© 2026 Ayush Singh
548 Market Street PMB 72296, San Francisco, CA 94104
Unsubscribe

Get the app Start your Substack